



MediCare Pro

Encryption / Decryption Application with an Integrity Monitor

CSG3101 – APPLIED PROJECT

GROUP D

Samod KANKANMGAMAGE	-	10598083
Chanuth WICKRAMASINGHE	-	10566806
Kavindu PITIGALAGEWAGE	-	10591304
Nivedha RAVINDRA	-	10598256
George RANASINGHEARACHCHILAGE	-	10625799



Abstract

Safeguarding the Sensitive patient's data has paved the way for a heated debate. This report outlines "MediCare Pro", an Encryption/Decryption Application with an integrity monitor tailored for hospitals. It provides a secure platform for doctors to both encrypt and decrypt patients report, incorporating robust integrating monitor system. The graphical interface mandates use authentication, with secured stores credentials. Utilizing Advance Encryption Standard (AES) encryption algorithm, the application encrypt file, and WinSCP ensures securing file sharing among Doctors. An essential feature involves an Administrative backend powered by PowerShell, monitors file integrity. Anomalies trigger alerts to doctors via HTML messages, ensuring a secure medical data exchange process. With a focus on simplicity and security, the application provides a streamline experience for doctors handling sensitive patient's data.

Table of Contents

CHAPTER 1: INTRODUCTION.....	3
1.1 CHAPTER OVERVIEW	3
1.2 BACKGROUND	3
1.3 PROBLEM	4
1.4 PROJECT AIM	4
1.5 PROJECT SCOPE	4
1.6 SCHEDULE	5
1.7 PROJECT DELIVERABLES	6
1.8 TEAM CAPABILITIES.....	7
1.9 TOOLS AND TECHNICAL REQUIREMENTS	8
1.9.1. Hardware Requirements	8
1.9.2. Software Requirements	8
1.9.3. Skills Requirements	8
1.9.4. Data Requirements	8
1.10 CHAPTER SUMMARY	9
CHAPTER 2: LITERATURE REVIEW.....	10
2.1 CHAPTER OVERVIEW.....	10
2.2 PROBLEM DOMAIN	10
2.3 RELATED WORK	12
2.4 TECHNOLOGICAL REVIEW	15
2.5 EVALUATION AND BENCHMARKING.....	16
2.6 CHAPTER SUMMARY	16

CHAPTER 3: SOFTWARE REQUIREMENTS SPECIFICATION	17
3.1 CHAPTER OVERVIEW.....	17
3.2. REQUIREMENTS ELICITATION METHODS	17
3.2.1 Literature Review	17
3.2.2. Brainstorming within the team	17
3.2.3. Experiment – Prototyping.....	18
3.3. DATA COLLECTION AND ANALYSIS	18
3.4 CONTEXT DIAGRAM	20
3.5 USE CASE DIAGRAM	21
3.6 USE CASE SPECIFICATION	21
3.7 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS	23
3.8 APPLICATION DIAGRAM.....	25
3.9 CHAPTER SUMMARY	25
CHAPTER 4: DESIGN SPECIFICATION	26
4.1 CHAPTER OVERVIEW.....	26
4.2 DESIGN GOALS	26
4.3 SYSTEM ARCHITECTURE DESIGN	27
4.4 SYSTEM DESIGN.....	28
4.4.1 Component Diagram	28
4.4.2 Sequence Diagram.....	29
4.4.3 UI Design.....	30
4.5 CHAPTER SUMMARY	32
CHAPTER 5: IMPLEMENTATION	33
5.1 CHAPTER OVERVIEW.....	33
5.2 TECHNOLOGICAL SELECTION	33
5.2.1. Encryption and Decryption Software with Python.....	33
5.2.2. Advanced Encryption Standards (AES).....	34
5.2.3. Integrity Monitor (PowerShell).....	36
5.2.4. SFTP Protocol (Windows Server 2019)	37
5.2.5. Message Communication (IIS and Server.js)	39
5.3 IMPLEMENTATION OF CORE FUNCTIONALITIES	42
5.4 CHAPTER SUMMARY	46
CHAPTER 6: TESTING.....	47
6.1 CHAPTER OVERVIEW.....	47
6.2 OBJECTIVES AND GOALS OF TESTING	47
6.3 FUNCTIONAL TESTING.....	47
6.4 NON-FUNCTIONAL TESTING.....	49
6.5 CHAPTER SUMMARY	50
CHAPTER 7: CONCLUSION	50
7.1 CHAPTER OVERVIEW.....	50
7.2 ACHIEVEMENTS OF RESEARCH AIMS & OBJECTIVES.....	50
7.3 UTILIZATION OF KNOWLEDGE FROM THE COURSE.....	51
7.4 USE OF EXISTING SKILLS.....	51
7.5 USE OF NEW SKILLS	52
7.6 ACHIEVEMENTS OF LEARNING OUTCOMES	53
7.7 PROBLEM AND CHALLENGES FACED.....	54
7.8 DEVIATIONS	54
7.9 FUTURE ENHANCEMENTS.....	56
7.10 CHAPTER SUMMARY	57
8. TEAMWORK.....	58
8. REFERENCES	61

Chapter 1: Introduction

1.1 Chapter Overview

Chapter 1 emphasizes the significance it is for medical data communication to be safe in light of recent developments in medical technology. The project presents a solution through encryption / decryption and integrity monitoring with a focus on protecting off cyber-attacks and ensuring patient data integrity. It offers a comprehensive overview by outlining the project's scope, team members' capabilities, and a project schedule. Additionally, tools and technical requirements along with hardware and software requirements, are highlighted.

1.2 Background

In this current day with technological advancements the medical industry is adapting and transforming its ways to exchange its sensitive information. Medical practitioners and specialists attempt to strengthen the security of patients' data. In relation to this crucial need to safeguard patient records, the authors has decided to bring up a solution to implement a secure medical data exchange through Encryption and Integrity monitoring.

The sensitivity of patient data linked with data breaches has brought about the critical requirement for safe communication in the medical industry (Satar et al., 2021). The impact of cyber threats has rapidly increased hence resulting in many serious consequences where patients' health has become critical, patient privacy hindered, patient distrust, misdiagnosis and including mistaken treatment (Shahzadi et al., 2019).

The decision to create this crucial project is by the commitment to undertake the challenges faced by the healthcare and provide a solution for all the patients with utmost care. This application will not only address the confidentiality and integrity of medical data but also enable efficient data transmission among health professionals.

To create this crucial project comes from the understanding that need to use Secure File Transfer Protocol (SFTP) which is known for its encryption and authentication, by using SFTP the authors can ensure that the file data transfers are secure, The toolkit will integrate SFTP to ensure the transmission of patient data is done through a secure environment.

Another fundamental component of The solution is the WinSCP Server which is carefully picked considering its security features. Additionally, digital signatures will be integrated for sender verification and integrity of data, this guarantees that the medical records received are not tampered with or altered.

1.3 Problem

The topic addressed in this study is fundamental and vast in its impacts. In today's rapidly changing healthcare industry, where technology is transforming how patient information is maintained and transmitted, ensuring the security and integrity of medical data has never been more vital. With the rising frequency and sophistication of cyber threats (Iqbal et al., 2019), the potential adverse effects of data breaches in the medical sector are serious, ranging from compromised patient care due to misdiagnoses and wrong therapies to violated patient privacy and broken trust. The relevance of this project comes from its commitment to mitigating these risks by providing a comprehensive solution that combines encryption and integrity monitoring. The initiative solves this issue by prioritizing patient record security through encryption and preserving data integrity through secure transfer protocols.

1.4 Project Aim

The primary objective is to design, develop, and evaluate an Encryption/Decryption Application for a hospital, along with an Integrity Monitor which will facilitate in encrypting and decrypting patient information, ensuring the integrity of sensitive patient data while they are transmitted between doctors and specialists.

1.5 Project Scope

The Encryption/Decryption application will enable doctors to encrypt patients' reports and transmit these encrypted reports via the Secure File Transfer Protocol (SFTP). As SFTP is access-controlled, only authorized doctors can gain entry. Once the files are transmitted, specialists can decrypt them using the Encryption/Decryption application. As a result, the application will ensure the confidentiality, integrity, and of patients' information.

1.6 Schedule

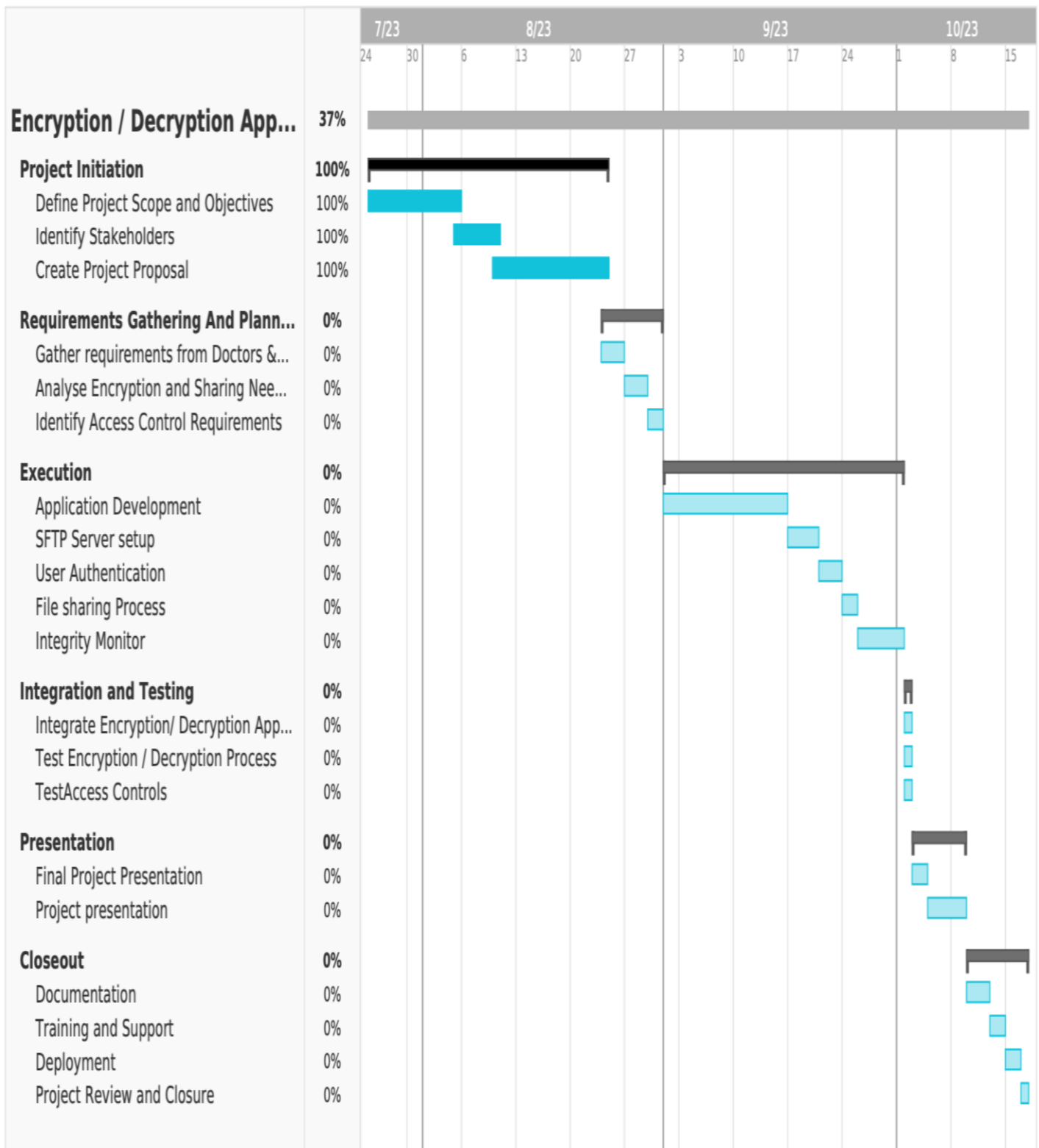


Figure 1: Gantt Chart

1.7 Project Deliverables



: Completed within the given timeline



: Could not Completed within the given timeline

No.	Dates	Deliverables
1	30/07/2023	Problem Identification
2	13/08/2023	Initial Literature Review
3	20/08/2023	Initial version of Proposal
4	24/08/2023	Final Project Proposal (Assignment 01)
5	06/09/2023	Detailed Literature Review
6	10/09/2023	Software Requirements Specification
7	12/09/2023	System Design
8	15/09/2023	First version of Prototype (preprocessing and Feature Engineering)
9	05/09/2023	Second version of prototype (Model building, Training and Testing)
10	07/10/2023	Testing results (List of issues identified)
11	09/10/2023	Project Presentation (Assignment 02)
12	12/10/2023	Final Version of Prototype (Complete prototype with improvements)
13	20/10/2023	Final Documentation (Thesis)
14	20/10/2023	Project Final Output (Assignment 03)

Table 1: Project Deliverables

Throughout the development process, deviations were introduced not with the regard to the schedule, but specifically to the architecture and mechanism of the application. A detailed explanation of the following deviations is discussed in [7.8 Deviations](#) topic in the latter part of the report.

1.8 Team Capabilities

The below table illustrates teach team member's capabilities and skills.

CS- Computer Science
Cyber – Cyber Security

Capabilities	Samod (CS)	Chanuth (CS)	George (Cyber)	Mihiran (CS)	Nivedha (Cyber)
Leadership and project Management		✓			
Good Communication skills	✓	✓	✓	✓	✓
Research Skills	✓	✓	✓	✓	✓
Excellent Report Writing Skills	✓	✓	✓	✓	✓
Excellent Coddng skills (Python)	✓	✓	✓	✓	✓
Excellent knowledge of Cyber security concepts	✓	✓	✓	✓	✓
Good problem-solving Skills	✓	✓	✓	✓	✓
Time Management	✓	✓	✓	✓	✓

Teamwork and collaboration	✓	✓	✓	✓	✓
Creativity and Innovative	✓	✓	✓	✓	✓

1.9 Tools and Technical Requirements

1.9.1. Hardware Requirements

- **Minimum of 8 GB RAM** to Manage larger volume of Patients records.
- **Minimum Intel Core i5Processor** to run the application.
- **Minimum hardware disk of 100 GB** to Store necessary data.

1.9.2. Software Requirements

- **Windows server 19** – To run programs such as Python and other required tools
- **WinSCP** – To access SFTP protocol
- **Python** – The programming language used to develop the application
- **MS Office** – For the documentation of the project
- **Google Drive** – To back up all the project files and data.
- **Git/GitHub** – To back up the application code

1.9.3. Skills Requirements

- Programming
- Researching skills
- Good communication skills
- Problem solving skills
- Report Writing Skills

1.9.4. Data Requirements

- **Doctors Information**
Doctors information who are using this application, including their names, Contact details and any relevant identifiers (Doctor's ID)
- **Medical Data**
Files to be Encrypted /Decrypted (Dummy medical files for application testing)
File type (. PDF, txt, Doc)
File metadata (File size, File name)
- **Encryption Data**
Encryption Keys
Encryption algorithms

- **Integrity Check**
Hash Values
Verification results (Passed/Failed)
- **SFTP Server Data**
SFTP server details (Host, Port)
SFTP configure details
- **Error and Exception Data**
Information's about errors and exceptions that can occurs while using the application
- **Documentation and Help data**
User guides
Manuals

1.10 Chapter Summary

Within the chapter 1, the authors provided an in-depth background related to security aspect of healthcare, highlighting the critical need for a solution for the identified problem and authors clearly defined the project scope and the aim. Furthermore, the chapter delved into project schedule, team capabilities, and the tools and technical requirements for the proposed application including, hardware, software, skills, and data.

Chapter 2: Literature Review

2.1 Chapter Overview

The chapter 2, dives into the problem domain of healthcare especially in sri Lankan context, exposing the vulnerabilities and major and minor issues that are currently facing in patient's data security and sharing. MediCare Pro stands out as the ultimate solution to overcome all vulnerabilities. Additionally, the chapter offers a overview of related work and a critical technology review and an evaluation presenting their cons and pros. This approach ensures that MediCare Pro integrate with finest features from each technology section, establishing it as a comprehensive solution

2.2 Problem Domain

Concerns over the security and exchange of private patient information and medical records have grown in the healthcare sector, particularly in Sri Lanka. Professionals in healthcare have turned to poor methods that put patients' private information at serious danger due to the lack of specialized software solutions and security systems. The authors examine the complex issues encountered by healthcare professionals and organizations that led to the creation of the Medicare Pro application. Moreover, an explnation of both significant and Minor Problems, describing how Medicare Pro successfully resolves each of them.

Major Problems

Lack of Data Security

The absence of effective data security procedures in healthcare facilities was one of the main problems that made Medicare Pro necessary. Without using sufficient encryption, medical professionals have been keeping patient records on their hard drives, laptops, and personal computers. This careless handling of data security is extremely alarming since it leaves private patient information, such as names, phone numbers, addresses, and vital medical information, open to unauthorized access.

Solution by Medicare Pro: Medicare Pro resolves this problem by using the Advanced Encryption Standard (AES) method to encrypt information, guaranteeing that private medical records will stay secure even in the event that a device is lost or compromised. Sensitive patient data will be protected thanks to its safe encryption.

Insecure File Sharing

Currently, transmitting patient reports means using unsecure channels like WhatsApp or email, as well as depending on third parties like nurses or assistants. Due to the lack of sufficient security measures to protect patient data during transmission, this strategy presents serious dangers.

Solution by Medicare Pro: With the help of the Secure File Transfer Protocol (SFTP), Medicare Pro provides a safe way to share files. This makes it possible for doctors to securely and easily share encrypted material with their other doctors (Senior/Specialists). This eliminates the possibility of unwanted access to private medical information by addressing the vulnerability of unprotected file sharing.

Data Integrity

It is essential to ensure the integrity of transferred medical files because any tampering could have serious medical impacts. However, there are currently no systems in place for medical experts to check the accuracy of sent patient data.

Solution by Medicare Pro: For sent files, Medicare Pro incorporates an integrity check capability. A hash function is used to accomplish this job, which enables medical professionals to confirm that the files weren't changed during transmission. The small but important issue of data integrity in healthcare is addressed here.

Minor Problems

Unauthorized Access

The problem of unauthorized access to patient information is ongoing. Unauthorized individuals run a significant danger of decrypting encrypted medical files if user authentication is insufficient.

Solution by Medicare Pro: User authentication is used by Medicare Pro to handle this issue and make sure that only permitted users can access the application. The privacy and confidentiality of patient data are successfully protected by this security precaution, which essentially prevents the risk of unauthorized individuals obtaining access to sensitive patient information.

2.3 Related work

Citation	Brief Summary of the work	Technology	Limitations	Contribution
(Hendi et al., 2019)	The research addresses the growing demand for cloud systems due to the increase cloud and diverse types of data being transmitted over internet and a need of robust encryption/decryption method to secure expanding applications and the data while being generated and transmitted. As a result, Simple and High Secure Encryption-Decryption (SHSED) is designed especially for cloud computing applications.	Logical XORing, addition, subtractions and byte shifting for encryption/decryption	Shows Slightly slower encryption/decryption efficiency compared to DES	A robust encryption/decryption method to secure expanding applications and the data while being generated and transmitted.
(Sumagita & Riadi, 2018)	The research paper addresses the security vulnerabilities available in web-based that are using MD5 Hash function for password encryption. However, in the study a proposed a upgrade version of SHA 512 which is more robust against potential attacks and especially collision attacks.	SHA 512 Hash Function	Need of existing user to update their passwords and potential performance can be impact due to the complexity of SHA 512	The primary contribution lies in fortifying eb application security y mitigating critical vulnerabilities. Additionally, the

			compared to MD5	shift from MD5 to SHA512 demonstrates a practical solution for cybersecurity challenges in the real world.
(Nakamura & Kuga, 2023)	The paper introduces MSCP, a novel file transfer tool designed for efficient large-scale data sharing in computing research. It enhances throughput by employing multiple SSH connections and achieving impressive performance gains in memory-to-memory copy experiments	MSCP, Secure Shell (SSH)	MSCP is still under Development	A user-friendly, high performance larger scale file transfer solution
(Orobosade et al., 2020)	The paper proposes a hybrid encryption algorithm combining symmetric and asymmetric cryptography to user's privacy and security in the cloud. It uses AES to encrypt data before storage and ECC combined with AES to ensure the confidentiality of the data.	AES and Elliptic Curve Cryptography (ECC)	If vulnerabilities discovered of AES or ECC, the entire algorithm would compromise.	A hybrid encryption model to secure Cloud application data
(Al-kateeb & Mohammed, 2020)	This paper proposes a new algorithm for encryption/decryption for audio files leveraging discrete Wavelet transform (DWT) and biometric features to ensure the high-level confidentiality and reliability. In order to increase the efficiency of the encryption process the algorithm uses numbers of hand geometry measurements as keys combined with DWT	Discrete Wavelet Transform (DWT) and biometric features	Changes in hand geometry overtime due to external factors can affect the accuracy and the reliability of the biometric based encryption.	A robust algorithm to encrypt/ decrypt audio files while ensuring the quality
(Shahzadi et al., 2019)	This journal discusses the importance of secure remote health monitoring, in which patient data is communicated to distant clinical professionals. In their study they proposed the RC5 Algorithm as an encryption method, however this is vulnerable to attacks, therefore this paper proposes a chaotic-based key scheduling algorithm,	RC5 Algorithm and chaotic-based key scheduling algorithm.	Need more security enhancement of RC5 and RC6 algorithms using higher order chaotic maps and security of clinical signals through optical chaos.	Their innovation ensures better protection of sensitive clinical information in remote health monitoring scenarios.

	which enhances the security of RC5 algorithm by robust chaotic key generation mechanism.			
(Iqbal et al., 2019)	The research focuses on securing patients' health records and introducing an innovative key management technique. It aims to create a lightweight key management system; this includes key distribution and key adjustment when a user joins or leaves. The aim is to use asymmetric cryptography keys to distribute the group key and private keys to all members.	RSA-based public key cryptosystem and CRT Algorithm	The research has not been tested in a real-world environment and no prototyping.	For e-health applications in wireless mobile contexts, a safe and privacy-preserving 'key management method' is proposed.
(Gautam et al., 2019)	The research is aimed at securing confidentiality of data (EHR) stored in cloud environment. The method used are obfuscation and RSA for confidentiality and authentication, a ciphertext and secret key is generated in the process.	RSA algorithm and Obfuscation	Confidentiality can be increased	RSA provides both confidentiality and authentication, whereas obfuscation protects against reverse engineering and makes program or data more difficult for an adversary to decipher.
(Satar et al., 2021)	This paper introduces a Cloud-based Secure Healthcare Framework (SecHS) which is designed to secure medical data. It enhances Ciphertext Policy Attribute-Based Encryption (CP-ABE) and it incorporates privacy and integrity in the cloud environment.	AES for encryption. And MD5 algorithm for hashing	This research has not yet been implemented in the real-world healthcare.	Model security analysis for Cloud security requirements covers privacy, integrity, and access control.
(Sani, 2022)	This study proposes secure file encryption and decryption by merging AES and RSA algorithms. The study focuses mostly on AES for symmetric encryption and RSA for asymmetric encryption. The paper proposes combining the strengths of AES's speed and RSA's robust key management to improve file	AES and RSA	This research is mainly only about the effectiveness of the algorithms.	The new algorithm, which combines the features of two algorithms, will address the issue of key movement in a symmetric encryption

	encryption by combining the strengths of both methods.			algorithm as well as the issue of asymmetric encryption algorithm slowness.
--	--	--	--	---

Table 3: Related Work

2.4 Technological Review

Technology	Pros	Cons	Citation
Simple and High Secure Encryption-Decryption (SHSED)	Faster encryption / decryption efficiency	Slightly slower encryption/decryption efficiency compared to DES	(Hendi et al., 2019)
SHA 512 Hash Function	More Resistance for potential and especially collision attacks.	performance can be impact due to the complexity	(Sumagita & Riadi, 2018)
MSCP, Secure Shell (SSH)	High performance larger scale file can be transfer	MSCP is still under development	(Nakamura & Kuga, 2023)
AES combined with ECC	A hybrid encryption algorithm to ensure both confidentiality and security of the cloud data	Possibility of vulnerabilities found	(Orobosade et al., 2020)
Discrete Wavelet Transform (DWT) and biometric features	A new and a secured algorithm to encrypt/ decrypt audio files while ensuring the quality	Changes in hand geometry overtime can affect to the bio metric-based encryption.	(Al-kateeb & Mohammed, 2020)
RC5 algorithm	Fast encryption, low memory requirements, simple and secure	Can be broken by attacks due its poor diffusion.	(Shahzadi et al., 2019)
RSA-based public key cryptosystem and CRT Algorithm	Increase computational speed	The algorithm is nth yet test in real world environment conditions	(Iqbal et al., 2019)
RSA algorithm and Obfuscation	Obfuscation can increase computational efficiency, RSA secures the data, it fulfils confidentiality and authentication	RSA takes a long time for encryption	(Gautam et al., 2019)
MD5 and AES	MD5 is efficient and faster.	The algorithm is not yet implemented in real world.	(Satar et al., 2021)
AES and RSA	Faster and key movement is better	Only the effectiveness is considered rather than the security and other asepts	(Sani, 2022)

Table 4: Technology Review

2.5 Evaluation and Benchmarking

2.6 Chapter Summary

In this chapter, the vulnerabilities existing in healthcare sector in Sri Lanka was detailedly demonstrated. Especially, since there is no currently used process to share patient's data rather than depending on an online platform or a third person, with the solution offered from MediCare Pro to share files through SFTP protocol make it's the ideal solution.

Chapter 3: Software Requirements Specification

3.1 Chapter Overview

The Chapter 3 explores into the requirements elicitation methods including Literature review, collaborative brainstorming session with project team, and hands-on experimentations through prototyping. Additionally, finding from data collection and analysis bridge the theory to practice enabling the application to align with real-world healthcare needs. To visually present the functionalities and interactions a Use case diagram is used while the context diagram represent how system interacts with external entities. Finally, after detailed both functional and non-functional requirements along with user specification an overview diagram of the application is presented.

3.2. Requirements Elicitation methods

3.2.1 Literature Review

The Literature Review played a key role in laying a solid foundation of knowledge and comprehension of the current technologies, processes, and best practices in medical data security, encryption techniques, and integrity monitoring systems. The authors were able to acquire important insights on well-established requirements methodologies, emerging trends, technologies, algorithms, and possible challenges by methodically going through peer-reviewed journals and academic papers. This methodical evaluation not only made sure that the project was based on a strong theoretical foundation but also gave the team the opportunity to tap into the collective knowledge of the development of software. The literature review process played a key role in determining the course of the project and eventually helped create a strong and well-supported set of software requirements. This method ensured that the project was not only well-informed by academic discourse but also poised to address the real-world challenges facing secure medical data interchange.

3.2.2. Brainstorming within the team

The team brainstorming sessions proved to be a very effective way to extract software requirements for the project. However, thanks to this collaborative approach the team were able to draw on the combined knowledge, innovative thinking, and different ideas of the team members. The Authors came up with a wide range of concepts, potential features, and

functionality requirements for the secure medical data exchange software through organized brainstorming sessions. Moreover, multiple possibilities and possible challenges because of the collaborative aspect of these sessions were examined, which encouraged open discussion. Additionally, it promoted a sense of shared accountability and responsibility among team members, ensuring that every detail of the software requirements was carefully examined. The authors were able to improve and prioritize the needs in a methodical way by using the potential of teamwork, which eventually resulted in a more thorough and reliable project. This approach was crucial in ensuring that the final solution maintained to the highest levels of data security and integrity while meeting the demanding requirements of the health care industry.

3.2.3. Experiment – Prototyping

Another key technique for eliciting software needs in the project was experimentation through prototyping. The team decided to take a hands-on approach since the complexity of the medical data security was and how important the user experience was identified. The authors were able to turn abstract ideas into observable, interactive models by building functioning prototypes. These prototypes helped the team to evaluate and improve the needs in a real-world setting in addition to serving as visual representation. Additionally, a lot about user preferences, system usability, and possible areas for development through iterative testing and feedback cycles were learned. This technique made it possible to take a dynamic and flexible approach to gathering requirements, ensuring that the final product was both functional and user-centric. Furthermore, prototyping also gave the team an actual method to communicate, which helped to create a shared understanding of the desired outcome. By using this technique, the authors were able to match the software needs with actual user expectations, creating a secure medical data exchange system that is more efficient and user-friendly.

3.3. Data collection and analysis

a. LR

LR Findings	Citation
Due to the increase of cloud data transmitting, the authors proposed a robust encryption/decryption method to secure expanding applications and the data while being generated and transmitted.	(Hendi et al., 2019)
due to the security vulnerabilities available in web-based that are using MD5 Hash function for password encryption.	(Sumagita & Riadi, 2018)

the authors proposed an upgrade version of SHA 512 which is more robust against potential attacks and especially collision attacks.	
A novel file transfer tool designed for efficient large-scale data sharing	(Nakamura & Kuga, 2023)
A hybrid encryption algorithm combining symmetric and asymmetric cryptography to user's privacy and security in the cloud	(Orobosade et al., 2020)
A new algorithm for encryption/decryption for audio files	(Al-kateeb & Mohammed, 2020)
Since RC5 Algorithm is vulnerable to attacks, the authors proposed chaotic-based key scheduling algorithm for better security.	(Shahzadi et al., 2019)
The authors wanted to secure patient records, they found that their innovative key management technique, which consists of asymmetric cryptographic key: key distribution and key adjustment can secure patient records.	(Iqbal et al., 2019)
The authors found that using RSA, obfuscation, secret key and ciphertext can secure the confidentiality of E-Health records in the cloud.	(Gautam et al., 2019)
The authors researched and found out that their framework (SecHS) can secure medical data with privacy and integrity in the cloud.	(Satar et al., 2021)
The researcher suggests that using both AES's speed with RSA's key management can improve file encryption.	(Sani, 2022)

Table 5: LR Findings

b. Brainstorming

Criteria	Findings
Encryption Algorithm - AES	It was discussed and concluded that AES was much more secure and efficient compared to other encryption algorithms.
Secure File Transfer Protocol (SFTP)	As concluded in the brainstorming session, the authors decided to use SFTP over other protocols like FTP, in terms on security, authentication and integrity, this protocol is the best.
Integrity monitor	The authors understood that including an integrity monitor can help the receiver (specialist) to find if the file has been tampered with or not.
Hashing algorithm – SHA 256	In the knowledge and research done, this hashing algorithm is more computationally intensive and faster compared to other algorithms and it is highly secure.

Programming Paradigm - Object-Oriented Programming (OOP)	OOP is used because it can collaborate in the development, easy debugging, maintenance, and code reusability.
Design Methodology - Object-Oriented Analysis and Design (OOADM)	Debugging is straightforward, collaborative development, objects can be re-used.

Table 6: Brainstorming

c. Experiment/Prototyping

Criteria – feature selection, platform selection

Finding – what was found and concluded.

Criteria	Findings
Encryption Algorithm - AES	While experimenting, it was found that AES was much securer than other algorithms.
Secure File Transfer Protocol (SFTP)	This protocol imposed a better security aspect than FTP in the prototype.
Integrity monitor	The integrity monitor was a good component in the prototype since it gave an authentication aspect to the receiver of medical records.
Hashing algorithm – SHA 256	When testing the prototype, it was concluded that SHA-256 was the best and most secure algorithm of all.
Programming Paradigm - Object-Oriented Programming (OOP)	
Design Methodology - Object-Oriented Analysis and Design (OOADM)	

Table 7: Experiment and Prototyping

3.4 Context Diagram

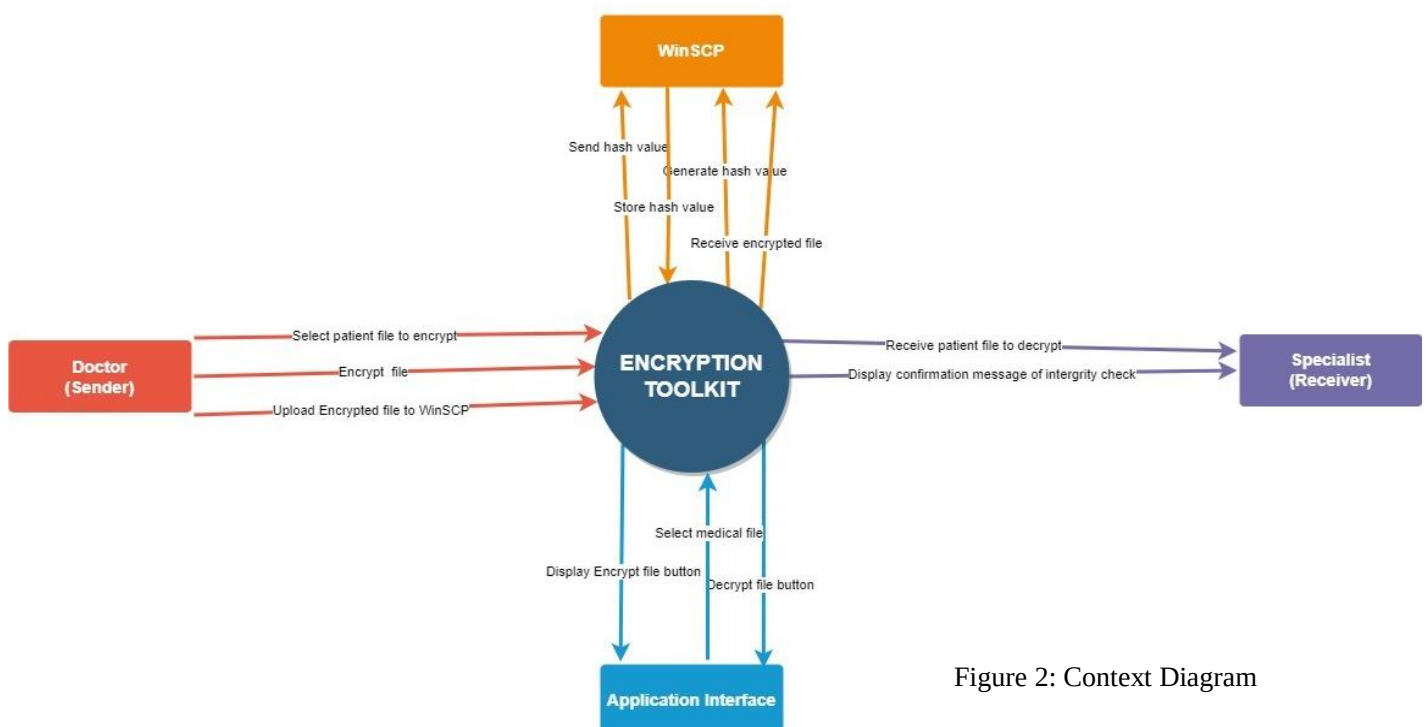


Figure 2: Context Diagram

3.5 Use Case Diagram

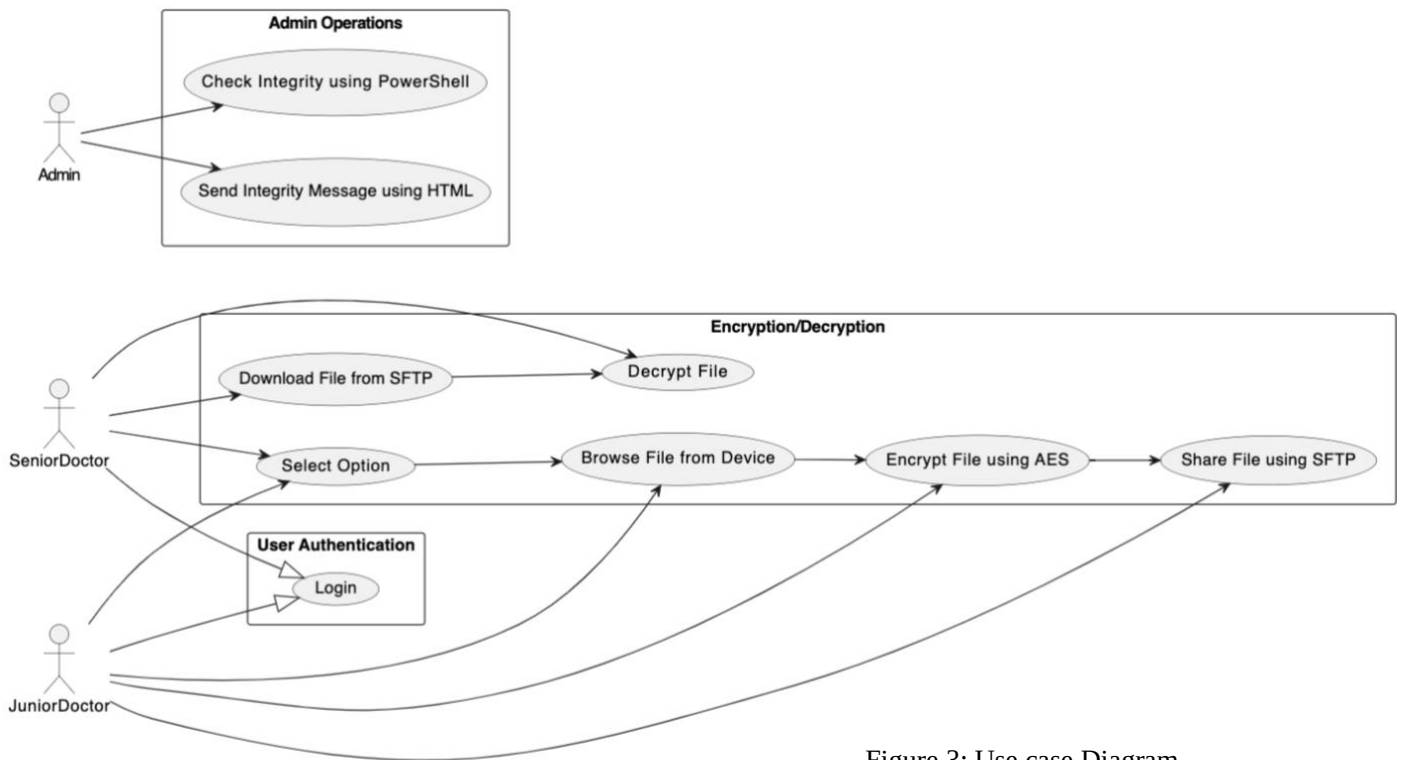


Figure 3: Use case Diagram

3.6 Use Case Specification

User Authentication

Use Case: User Login

Primary Actor: JuniorDoctor, SeniorDoctor

Description: This Usecase facilitate the JuniorDoc and the SeniorDoc to login to the system

Preconditions: The MediCare Pro application must be installed, and a User Account has already been created.

Main Flow:

1. The user selects the "Login" option.
2. The system prompts the user to enter their credentials.
3. The user enters the username and select enter.
4. The user enters the password and press enter.
5. The system validates the user.

Exceptions:

1. If the user credentials are miss match, the system will not give access to login to the application

Encryption/Decryption

Use Case: Secure File Handling

Primary Actor: JuniorDoctor, SeniorDoctor

Description: This use case includes the file encryption and decryption alone with the file sharing process.

Preconditions: The files must be saved in the Device

Main Flow:

1. The JuniorDoc selects encryption button.
2. Click browse the files:
3. The JuniorDoc browses and selects a file from their device.
4. The JuniorDoc press encrypt.
5. The system encrypts the file using AES (Advanced Encryption Standard).
6. The JuniorDoc can then choose to "Share File."
7. If "Download files" is chosen:
8. The SeniorDoc selects the file to download from the SFTP server.
9. The system downloads the file.
10. The SeniorDoc can then choose to "Decrypt File."

Exceptions:

1. In encrypting process, if the user selects an already encrypted file, the system will not again encrypt the file instead an error message will be printed.
2. In decrypting process, if the user selects aa already decrypted file (Plain file), a error message will be printed.
3. Once the file is encrypted only the share button will be enabled. Without encrypting files cannot be shared.

Admin Operations

Use Case: Integrity Checking and Messaging

Primary Actor: Admin

Description: This use case involves the admin checks for the integrity and notify the users

Preconditions: A file must be shared with through SFTP server to another doctor

Main Flow:

1. The admin selects the "Check Integrity using PowerShell" option.
2. The system performs integrity checks using PowerShell.
3. The admin can choose to "Send Integrity Message using HTML."

Exceptions:

1. If the file has/has not passed the integrity check, the admin can notify the respective doctor

3.7 Functional and Non-Functional Requirements

Functional Requirements

1. Data Encryption (Must Have):

- The software must incorporate robust encryption algorithms, such as AES-256, to safeguard the confidentiality of patient data during its transmission.

2. Data Decryption (Must Have):

- Only the authorized personals may be able to access this data and decrypt the encrypted data

3. Integrity Monitoring (Must Have):

- The software must ensure that originality in transferring the data, and if any modifications made it should inform the admin.

4. Secure File Transfer (Must Have):

- The software should ensure the security in transferring the encrypted data between the users. It should utilize a Secure File Transfer Protocol (SFTP)

5. User Access Control (Should Have)

- According to the role of the stakeholder in hospital they should be given access to the access to the data.

6. User Authentication (Must Have):

- Must be able to login with user credentials with username and password.

Non Functional Requirements

1. Performance (Must Have):

- The system should response to the user with its maximum response time in case of critical operation.

2. Reliability (Must Have):

- The availability of software must be in a higher level.

3. Security (Must Have)

- Security measures like encryption/ decryption and integrity monitoring, access control user authentication should meet the industry standards and follow the Heath Care Data Privacy Regulations.

4. Usability (Should Have):

- User interface should be intuitive and user friendly with clear navigation.

5. Maintainability (Could Have):

- Institute coding standards and documentation procedures to streamline future updates and maintenance processes.

6. Resource Constraint (Could Have):

- The software should be optimized to run considering the hardware and software constraints.

7. Interoperability (Could Have)

- Plan for integrate with the electronic health record system (EHR).

3.8 Application Diagram

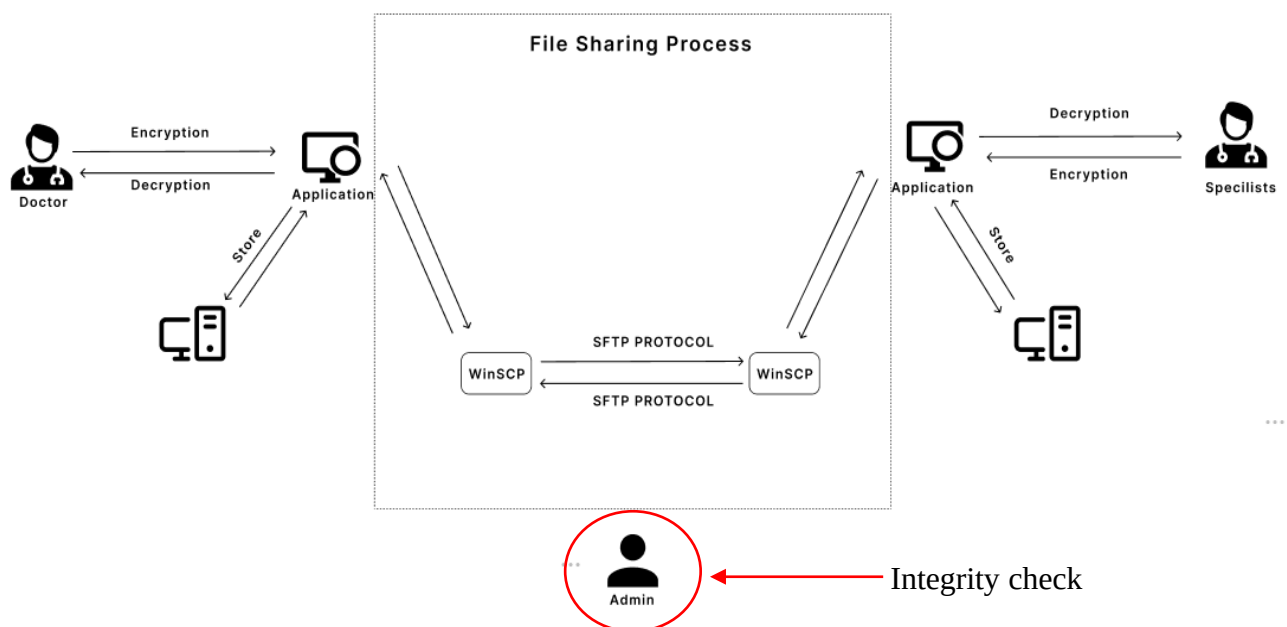


Figure 4: Application Diagram

3.9 Chapter Summary

In this chapter, the authors thoroughly demonstrated various requirements elicitation methods. The data collection and analysis combined with Literature review and brainstorming findings highlights the significant of a robust encryption algorithm and a secure file transfer protocol. Additionally, the importance of a user authentication and the integrity monitor is demonstrated. The chapter concludes with detailed context and Use case diagrams, as well as detailed functional and non-functional requirements, providing a strong foundation for the Implementation and testing chapters.

Chapter 4: Design Specification

4.1 Chapter Overview

Before the development stages of the application, it is necessary to create a software design and architecture. In this chapter, the authors describe the stages of the software design process, as well as the role of the software architect. The user interface (UI) design of the MediCare Pro application is intentionally kept simple and straightforward to enhance user-friendliness, particularly because the application is intended for use by doctors who are IT professionals. Given that the application is a peer-to-peer decentralized application, the software architecture consists of three tiers. Presentation tier, Application tier, and Data tier. The chapter also includes component, class, and sequence diagrams to illustrate and explain the software design in detail.

4.2 Design Goals

Before commencing the software design process, it is essential to outline a set of goals and objectives that the design aims to achieve. Taking these objectives into account leads to a formulation of a comprehensive design, that is capable of producing a working software solution with highest level of satisfaction. The main objectives of the application design are,

- Functional Suitability
- Maximum security
- Usability (User Experience)
- Reliability and Stability
- Performance efficiency
- Cost effectiveness

By aligning the design process with these above-mentioned objectives, the aim is to create an application that not only meet functional requirements but also excels in terms of security, usability, reliability & stability, performance, and cost effectiveness, ultimately ensuring the high level of satisfaction for the users.

4.3 System Architecture Design

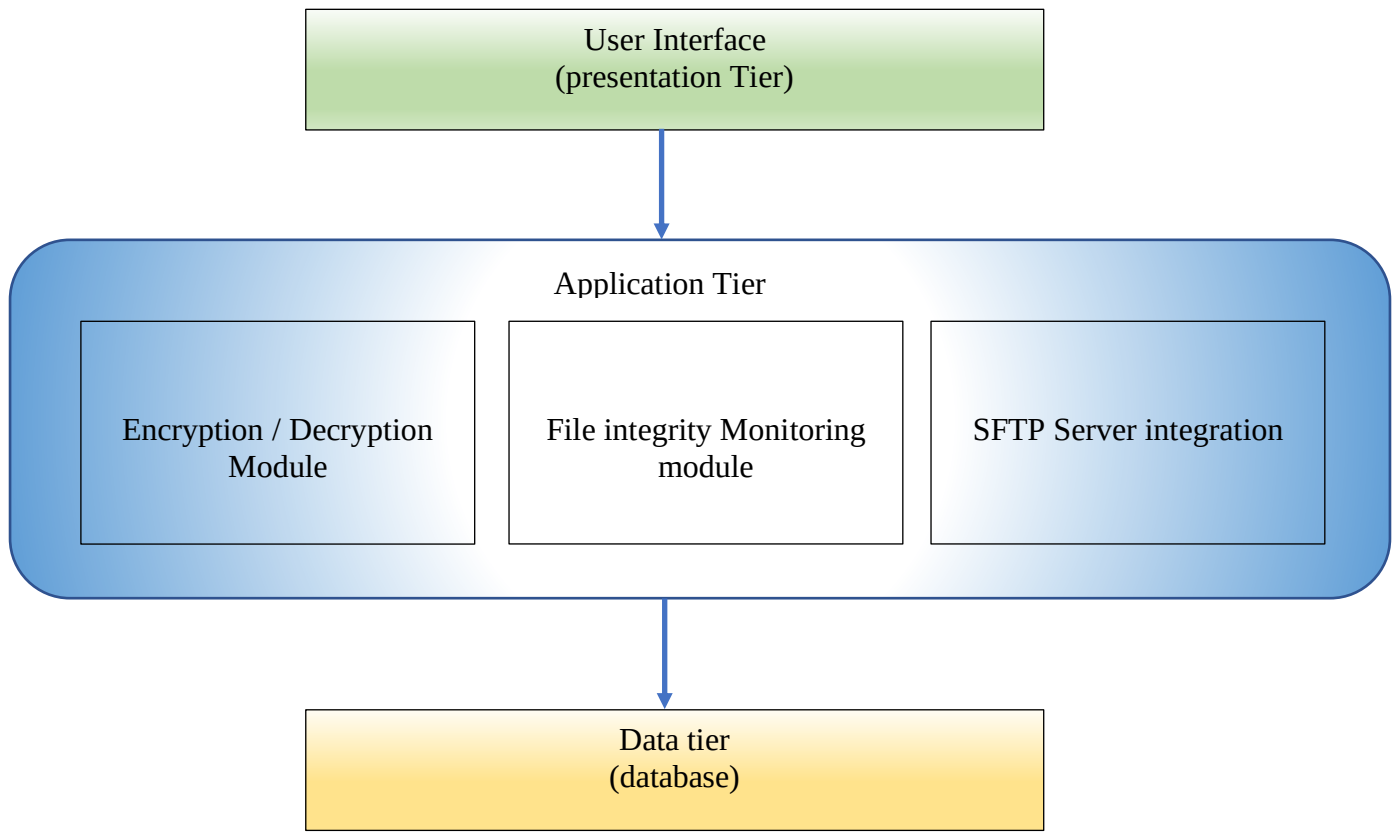


Figure 5: System Architecture Design

Presentation tier (User Interface) handles user interactions and provides a user-friendly interface for doctors to use the application. The Presentation tier interacts with the Application Tier, which consists of three main modules. Encryption and Decryption, File integrity monitor, and SFTP server Integration.

Encryption and Decryption Module: Responsible for encrypting and decrypting files using secure encryption algorithms (AES).

File Integrity Monitoring Module: Monitors the integrity of files by calculating and verifying checksums or digital signatures.

SFTP Integration: Manages the secure transfer of files to and from the SFTP server.

The Data Tier (Database) is used to store Generate hash values for documents in the application.

4.4 System Design

4.4.1 Component Diagram

The below diagram represents the component and entities of the system

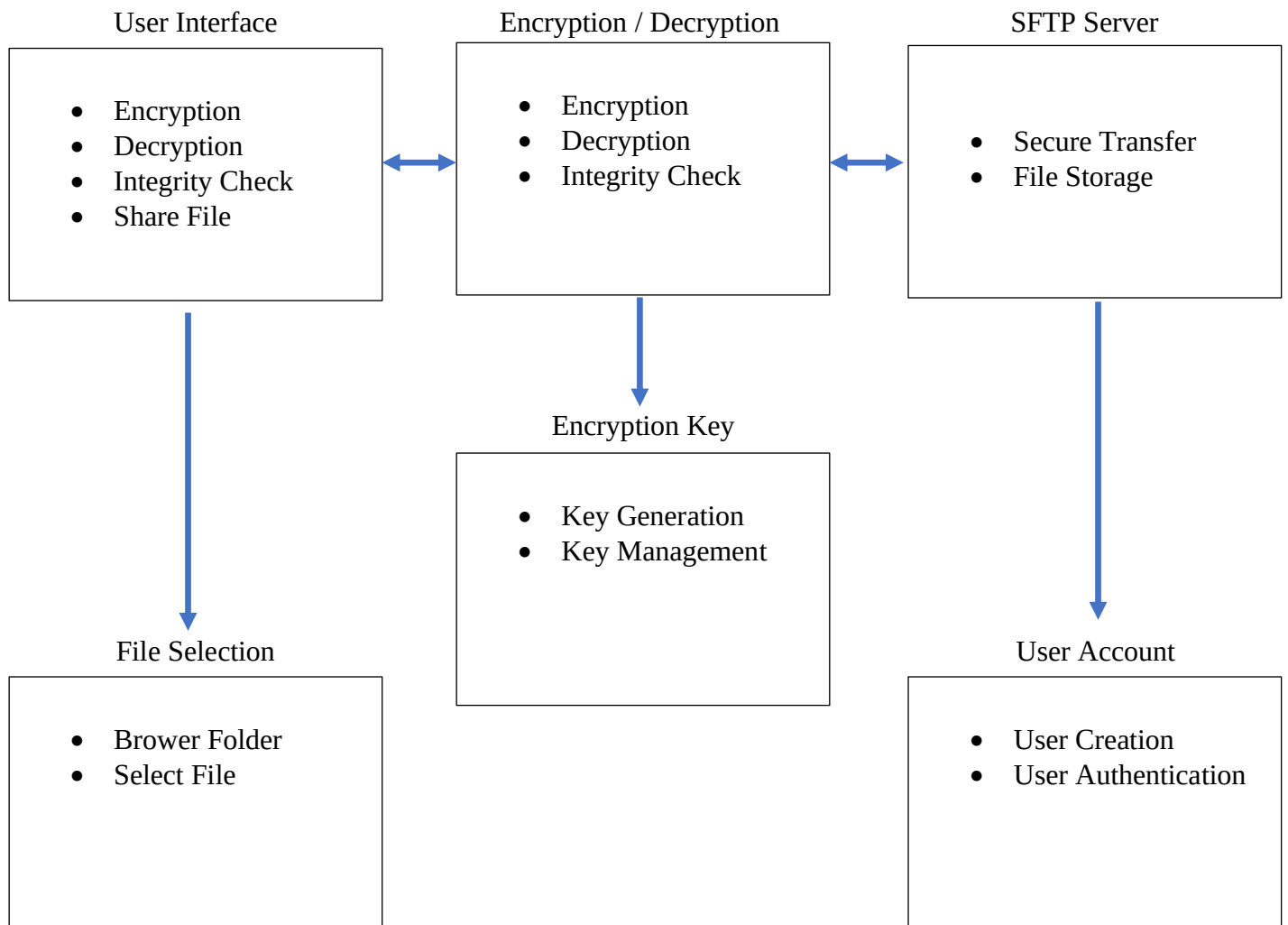


Figure 6: Component Diagram

4.4.2 Sequence Diagram

User A: Junior Doctor

User B: Senior Doctor / Specialist

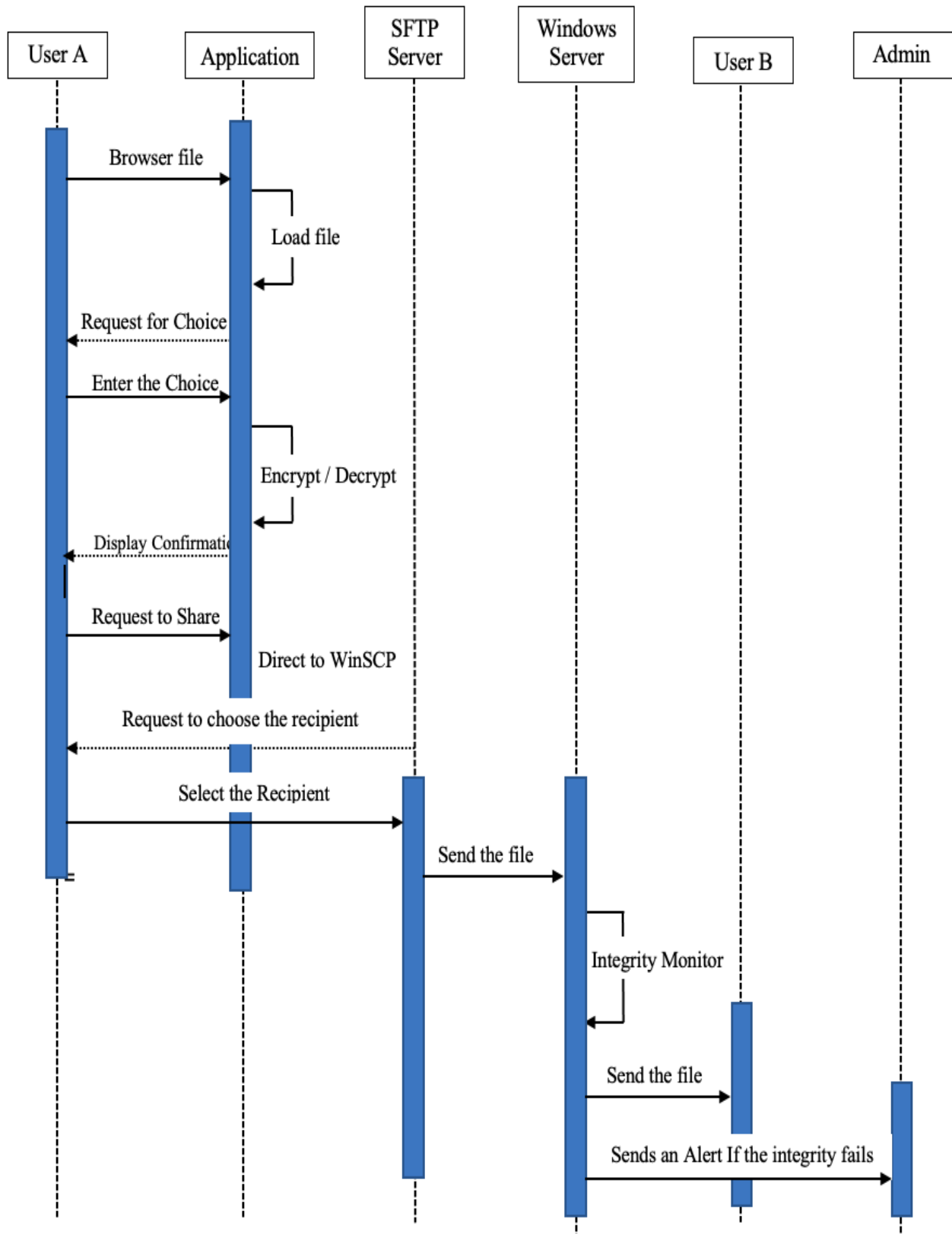


Figure 7: Sequence Diagram

4.4.3 UI Design

The following images shows the UI of the application. Figure 9 represent the splash Screen which indicates the hospital name, and the group name. Figure 10 represent the main menu of the application. It consists of the Options of the menu including Browser, encrypt and decrypt. Figure 7 & Figure 8 represent the User authentication window while Figure 11 and Figure 12 represent the Encryption and Decryption option windows.

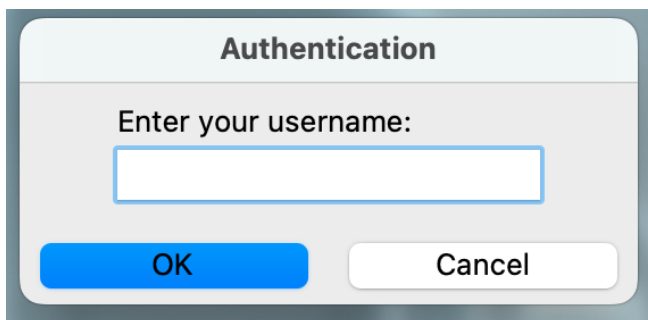
A dialog box titled "Authentication" with a light blue header. Below the header, it says "Enter your username:" followed by a white text input field with a blue border. At the bottom, there are two buttons: a blue "OK" button and a white "Cancel" button with a blue border.

Figure 8: User authentication (Username)

Figure 9: User authentication (Password)

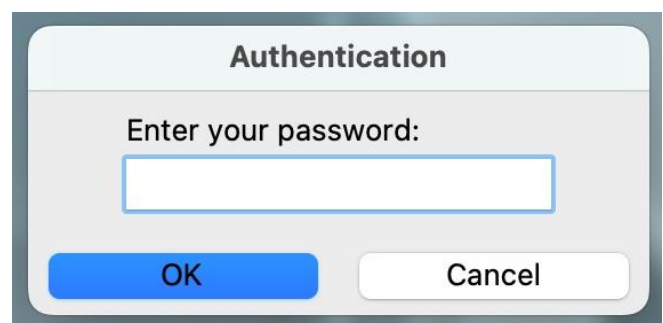
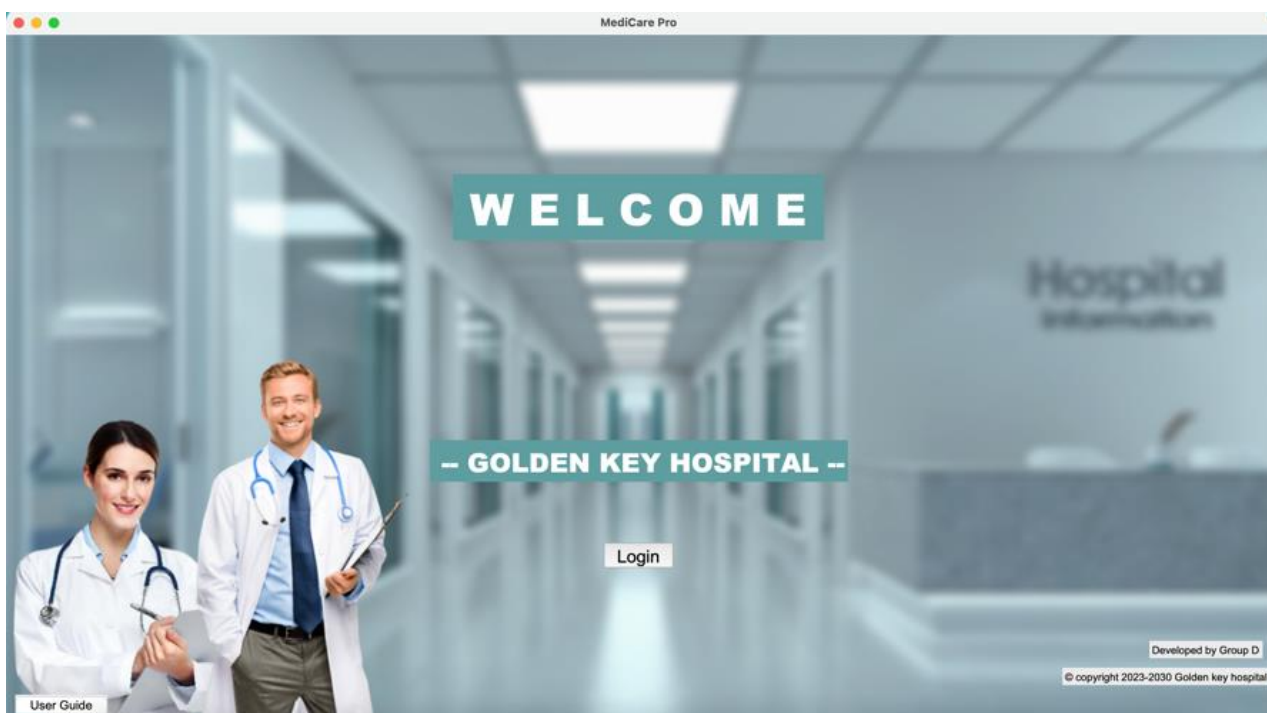
A dialog box titled "Authentication" with a light blue header. Below the header, it says "Enter your password:" followed by a white password input field with a blue border. At the bottom, there are two buttons: a blue "OK" button and a white "Cancel" button with a blue border.

Figure 10: Splash Screen (Welcome)

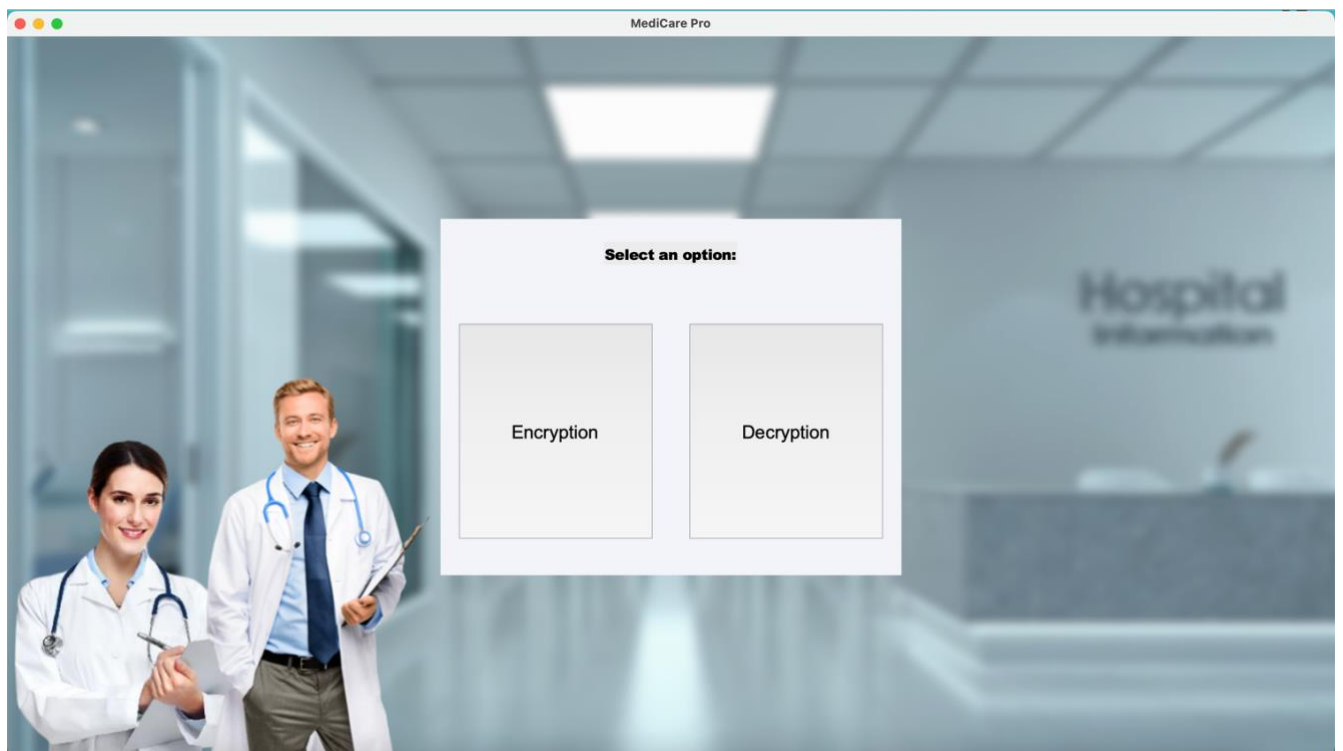


Figure 11: Main Menu

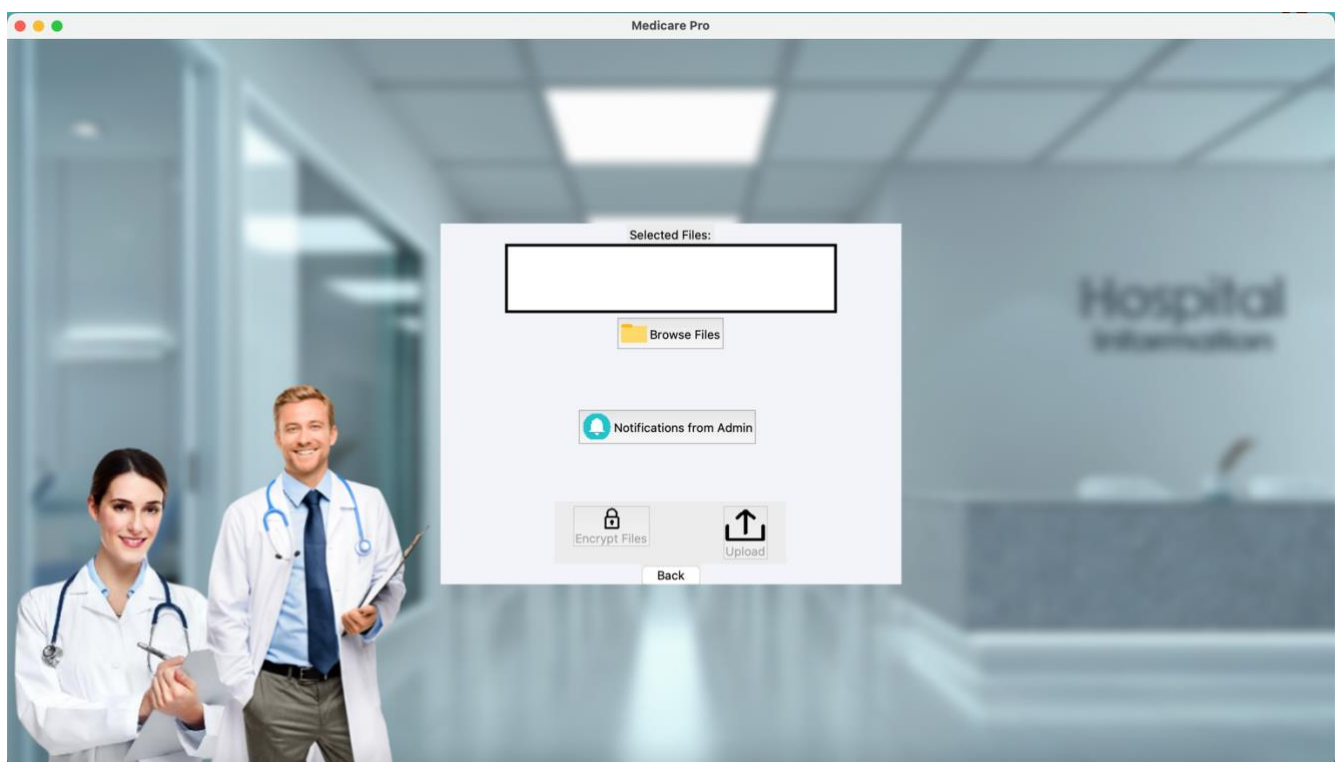


Figure 12: Encryption Window

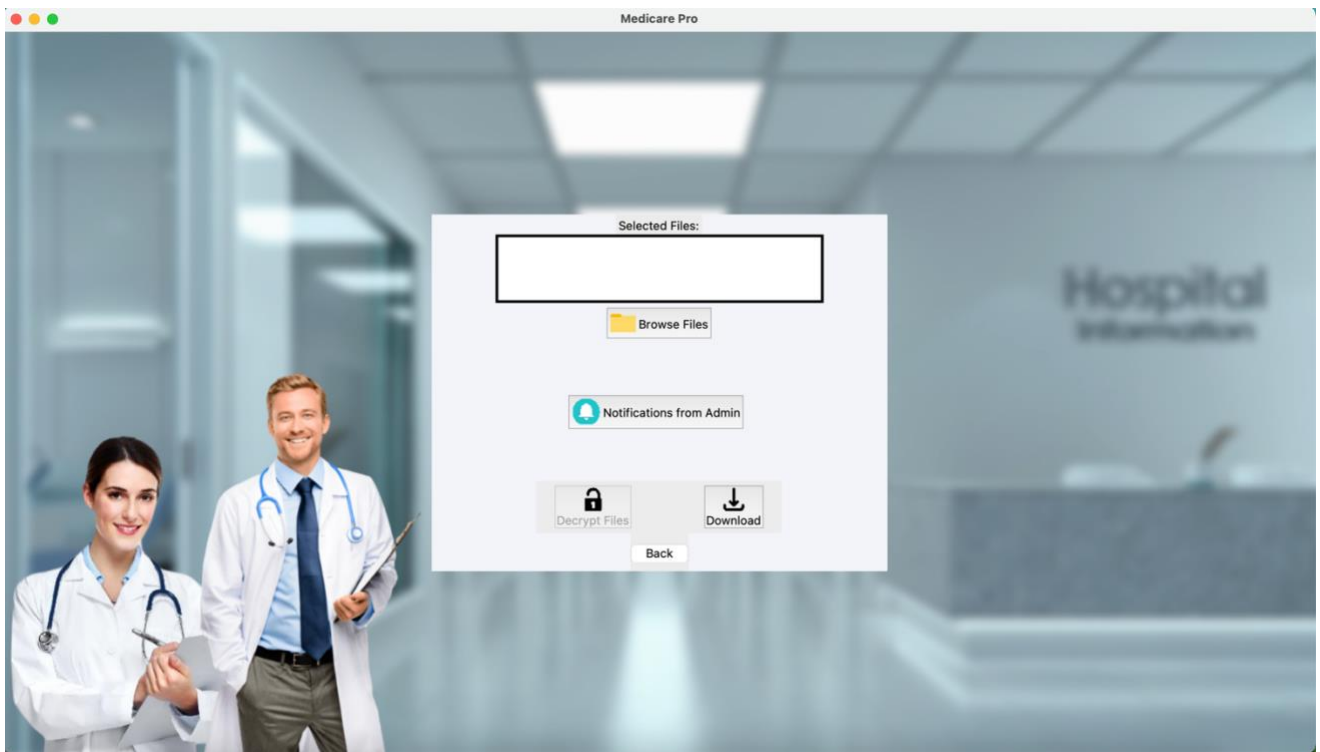


Figure 13: Decryption Window

4.5 Chapter Summary

In this chapter, the authors discussed the software architect and the software design by using component, class, and sequence diagram. Additionally, the actual UI design of the application were presented.

Chapter 5: Implementation

5.1 Chapter Overview

This Chapter demonstrate the strategic selection of technologies crucial for the success of the Medicare Pro application. The focus lies on encryption / decryption application built with python, Advanced Encryption Standard (AES), Integrity monitoring using PowerShell, Secure File Transfer Protocol (SFTP) on Windows Server 2019, and message communication through IIS and Server .js. Additionally, an integral aspect of this exploration delves into the implementation of core functionalities, specifically the secure file sharing process and the process of integrity monitoring, contributing to the overall robustness and effectiveness of the medical data exchange system.

5.2 Technological Selection

5.2.1. Encryption and Decryption Software with Python

The foundation of the project is safe data transfer through encryption and decryption. The authors choose Python as the programming language to do this. The encryption and decryption program were made possible by Python's strong libraries and simple integration. Data security is ensured by using a set of pre-determined random keys.

Why Python?

For a number of strong reasons, the authors decided to develop the encryption and decryption software in Python. Here some details about it.

Ease of Use and Readability - Python is well-known for being a simple and readable programming language, making it a great option for creating complex encryption methods. This readability was essential since it guaranteed that the team will be able to understand and maintain the code over the course of the project.

Rich Ecosystem of Libraries - The extensive ecosystem of libraries and modules available in Python significantly sped up the development process. The authors used the tkinter library for

the graphical user interface (GUI) in your particular implementation, subprocess to run other processes like WinSCP, Crypto. Cipher for cryptographic operations, and more. These libraries made development easier and guaranteed reliable functioning.

Cross-Platform Compatibility- Python is cross-platform, allowing it to function without significant changes across a range of operating systems. The ability to adapt to many contexts, including Windows Server 2019, MacOS, was crucial.

5.2.2. Advanced Encryption Standards (AES)

Why AES over Other Encryption Methods like DES?

Several factors led to the selection of Advanced Encryption Standard (AES) over competing encryption techniques like Data Encryption Standard (DES)

Security

Compared to DES, AES is seen to be safer. While AES has keys of 128, 192, or 256 bits, DES employs a key that is just 56 bits. The bigger key sizes of AES make brute-force assaults much more difficult, making it more resistant against cryptographic attacks.

Industry Standard

For symmetric encryption, AES has emerged as the industry standard. Governmental organizations, financial organizations, and the security community all recognize and use it. The security of AES is now more assured because to this standardization.

Efficiency

Performance-wise and in terms of memory use, AES is effective. It can swiftly encrypt and decode data, which is vital for real-time file encryption and decryption in a medical setting where responsiveness and speed are paramount.

Why a 256-Bit Key?

An additional level of protection is offered by selecting an AES key size of 256 bits. There are 2256 potential keys, which is an absurdly huge amount given the 256-bit key length. It is very

difficult for attackers to use brute-force techniques to crack encrypted data because to the enormous key space. Choosing a 256-bit key guarantees the highest degree of security, which is crucial when working with sensitive medical data even if 128-bit and 192-bit keys are still safe.

In conclusion, Python was picked because to its readability and wide range of library support, while AES was chosen due to its higher security, widespread use, and effectiveness. The security of the encryption and decryption software is further improved by the use of a 256-bit key size in AES, offering a strong defense against possible threats to the confidentiality of medical data. Figure 13 represent the python code for the AES encryption / decryption algorithm.

```
63
64 # Encryption part
65 def encryption_process(file_path, key):
66     cipher = AES.new(key, AES.MODE_EAX)
67     with open(file_path, 'rb') as file:
68         plaintext = file.read()
69     ciphertext, tag = cipher.encrypt_and_digest(plaintext)
70     with open(file_path + ".ecry", 'wb') as enc_file:
71         enc_file.write(cipher.nonce)
72         enc_file.write(tag)
73         enc_file.write(ciphertext)
74     os.remove(file_path)
75
76 # Decryption part
77 def decryption_process(file_path, key):
78     with open(file_path, 'rb') as enc_file:
79         nonce = enc_file.read(16)
80         tag = enc_file.read(16)
81         ciphertext = enc_file.read()
82     cipher = AES.new(key, AES.MODE_EAX, nonce=nonce)
83     plaintext = cipher.decrypt_and_verify(ciphertext, tag)
84     with open(file_path[:-4], 'wb') as dec_file:
85         dec_file.write(plaintext)
86     os.remove(file_path)
87
```

Figure 14: Python Encryption / Decryption Code

5.2.3. Integrity Monitor (PowerShell)

Why PowerShell?

Multiple strong arguments led to the decision to choose PowerShell as the language to create the Integrity Monitor.

Native Integration with Windows Server

Microsoft created the scripting language PowerShell exclusively for Windows systems. Due to its excellent integration with the Windows environment, it is an obvious option for activities requiring Windows Server. It was a sensible option for the project because of how simple it is to utilize on Windows Server computers.

Comparing Hash Values and Matching Integrity

Data integrity is crucially dependent on hash values. For a more thorough explanation of their relevance, read on. Hash values are one-of-a-kind digital signatures or checksums created from a file's content. The hash value differs noticeably when the file's content even slightly changes. Hash values are a trustworthy way to identify any file changes or manipulation because of this characteristic.

Comparing Hash Values

The hash values of files are computed and saved at the time of original upload or storage. The Integrity Monitor then updates the hash values of saved files at specified times or in response to particular occurrences. When a file's current hash value and its previously recorded hash value diverge, it indicates that the file went through some kind of modification.

Matching Integrity

To make sure that the integrity of files is maintained, hash values are compared. Maintaining data integrity is crucial when doctors are sharing patient records. The integrity of these data must be maintained since any alterations might have catastrophic repercussions. The Integrity Monitor's job is to find any discrepancies and tell administrators right away so they may take the necessary action.

The authors can quickly identify any unauthorized modifications to medical files by automating the production of hash values and regularly monitoring them using PowerShell. This adds another layer of protection and builds trust in the integrity of shared data. This guarantees the accuracy and integrity of the medical reports and information throughout transmission and storage, which is crucial in the medical sector. The figure 14 shows the PowerShell code used for the integrity monitor.

```

58 $files = Get-ChildItem -Path .\Files
59
60 # For each file, calculate the hash, and write to baseline.txt
61 foreach ($f in $files) {
62     $hash = Calculate-File-Hash $f.FullName
63
64     # Check if the file is in the baseline dictionary
65     if ($fileHashDictionary.ContainsKey($hash.Path)) {
66         $baselineHash = $fileHashDictionary[$hash.Path]
67
68         # Check if the file has been compromised
69         if ($hash.Hash -ne $baselineHash -and -not $compromisedFiles.ContainsKey($hash.Path)) {
70             # File has been compromised, notify the user and mark it as compromised
71             Write-Host "$($hash.Path) has modified!!!" -ForegroundColor Yellow
72             $compromisedFiles.Add($hash.Path, $true)
73         }
74     }
75     else {
76         # Check if the file is new
77         if (-not $newFiles.ContainsKey($hash.Path)) {
78             # A new file has been created, notify the user and mark it as new
79             Write-Host "$($hash.Path) has been created!!!" -ForegroundColor Green
80             $newFiles.Add($hash.Path, $true)
81         }
82     }
83 }
84
85 # Check if any baseline files have been deleted
86 foreach ($key in $fileHashDictionary.Keys) {
87     $baselineFileStillExists = Test-Path -Path $key
88     if (-Not $baselineFileStillExists) {
89         # One of the baseline files must have been deleted, notify the user
90         if (-not $compromisedFiles.ContainsKey($key)) {
91             Write-Host "$($key) has been deleted!!!" -ForegroundColor DarkRed -BackgroundColor Gray
92             $compromisedFiles.Add($key, $true)
93         }
94     }
95 }

```

Figure 15: PowerShell Code for Integrity Monitor

5.2.4. SFTP Protocol (Windows Server 2019)

Why Choose SFTP Over FTP

Data integrity and security were given considerable attention while choosing the Secure File Transfer Protocol (SFTP) over the more widely used File Transfer Protocol (FTP). Why SFTP was selected is as follows:

Enhanced Security

SFTP is essentially more secure than FTP. Compared to FTP, which transfers data in plain text, SFTP encrypts data during transmission, making it far less vulnerable to data interception and eavesdropping. In a medical context where patient data security is of the highest concern, this

level of protection is essential. Authentication SFTP makes use of secure authentication methods like SSH (Secure Shell) keys and passwords. By making sure that only authorized users can access the server and upload/download data, this reduces the chance of unauthorized access to confidential medical information. Data Reliability Data integrity checks are included into SFTP, guaranteeing that files are sent without modification during transmission. This is in line with the project's objective of protecting the accuracy of medical records. SFTP is compatible with firewalls since it uses a single port, often port 22. This lowers the need for broad port openings, simplifies network setup, and improves security by reducing attack avenues. Compatibility Windows, Linux, and macOS are just a few of the many systems that support SFTP clients and servers. This guarantees smooth file transmission and communication across various operating systems, which is important in a diversified medical setting.

Why Windows Server 2019?

The hosting platform for the SFTP protocol was chosen as Windows Server 2019 based on a number of important considerations. Platform for Business Use Windows Server 2019 is a powerful, business-grade operating system renowned for its reliability, security features, and broad support. Because of its ability to manage mission-critical activities, it is a dependable option for storing and maintaining private medical records.

Centralized user administration, authentication, and access control are made possible by Windows Server's Active Directory Integration feature, which seamlessly integrates with Active Directory. By limiting access to and sharing of patient data to just authorized healthcare providers, this feature is essential. Administrators with different degrees of competence may utilize Windows Server thanks to its user-friendly interface and well-known user interface. This is essential for healthcare organizations since IT staff could have a variety of skill sets.

As user and data demands in a medical setting increase, Windows Server's scalability is highly scalable to meet these needs. When the demands of the healthcare center change, it is simple to expand. Advanced security tools including Windows Defender, Credential Guard, and Just-In-Time Administration (JIT), which further improve the security of sensitive medical data, are included in Windows Server 2019's security features. Maintaining and updating Microsoft offers Windows Server a wide range of support services and frequent upgrades, ensuring that

security flaws are quickly fixed, and the system is always in line with the most recent security requirements.

To sum up, a strong dedication to data security, integrity, and dependability was the primary motivator for the selection of the SFTP protocol with Windows Server 2019. These technologies were regarded crucial in order to satisfy the demanding standards of the medical sector and guarantee the efficient and secure exchange of medical records among healthcare providers.

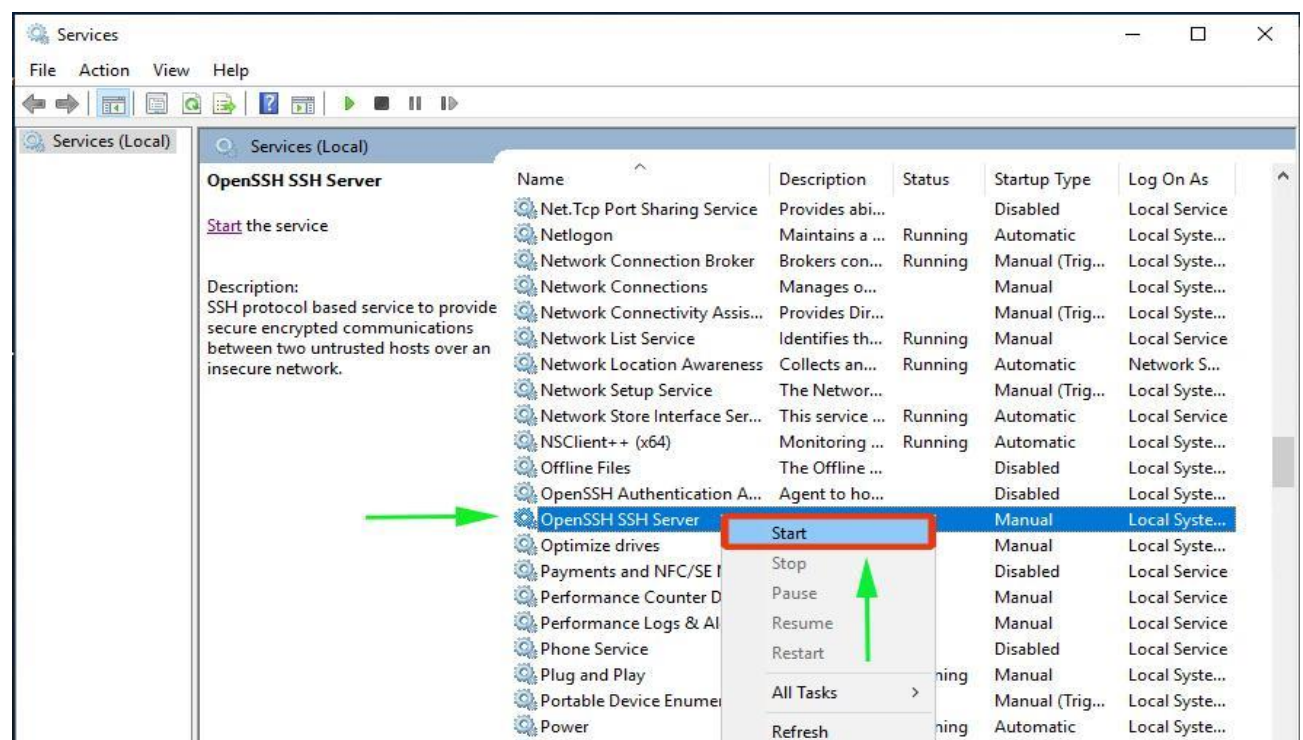


Figure 16: Open SSH server Start (SFTP)

5.2.5. Message Communication (IIS and Server.js)

Why JavaScript and Server.js?

JavaScript was chosen to build the server. JavaScript file was motivated by a number of elements that make it an appropriate option for managing real-time messages inside the project,

Real-Time Capabilities: JavaScript excels at managing real-time communication, especially when coupled with Node.js. This is essential for the messaging system since it enables real-time changes and message delivery.

Event-Driven design: The event-driven design of Node.js, a JavaScript runtime environment, is well-known. This makes it capable of managing several connections at once, which is essential for allowing communications between numerous users.

WebSocket: Over a single TCP connection, WebSockets, a crucial component of Node.js, provide full-duplex communication channels. They make it the best option for real-time messaging since they allow two-way communication between the server and clients.

Wide-ranging library support is provided through the Node Package Manager (npm), which is a rich ecosystem of libraries and modules for Node.js. The authors implemented WebSockets especially using the 'websocket' package, which made development easier and guaranteed stable communications.

[Internet Information Services \(IIS\)](#)

Microsoft created the web server software known as Internet Information Services (IIS). The following factors led to its selection as the location for the HTML files that make up the messaging system's user interface, IIS is a logical option for hosting web applications on Windows-based systems because of its strong integration with the Windows Server environment.

In order to facilitate user interactions in the messaging system, IIS's high scalability and capacity to manage many concurrent connections are critical. Strong security features, such as procedures for authentication and authorization, are provided by IIS. To guarantee that only authorized users may use the message system, this is essential. Ease of Configuration: Installing IIS is rather simple, and it can be set up to function with other project elements like Windows Server and the server.js application without any configuration issues.

Firewall Port Configuration (Port 3000)

The authors opened firewall port 3000 to allow both incoming and outgoing traffic so that the server.js application and client devices could communicate. This port setup accomplishes numerous crucial tasks:

- Opening port 3000 to incoming traffic makes it possible for the server.js application to watch for incoming WebSocket connections from clients. In order to create channels for bidirectional communication, this is required.
- To maintain the Outgoing Traffic in the server.js application uses outgoing traffic on port 3000 to transmit messages and changes to clients, ensuring that messages are delivered in real-time.
- Security handling by designating port 3000 for this use, the authors separate the messaging traffic from other network activity, boosting security and making firewall rule maintenance easier.
- Scalability handling By setting a particular port for The messaging system, the authors may scale it up in the future by adding more instances of the server. Is necessary, a load-balancing js application may be launched.

In conclusion, the decision to utilize JavaScript and Server.js for real-time messaging and Internet Information Services (IIS) to host the user interface gives a reliable and scalable alternative for promoting user communication inside the project. Messages are delivered and received safely and instantly thanks to the firewall port setup (port 3000). The Figure 16 shows the host file edit while the Figure 17 represent the firewall port open for notifications.



```
12 # lines or following the machine name denoted by a '#' symbol.
13 #
14 # For example:
15 #
16 #      102.54.94.97      rhino.acme.com      # source server
17 #      38.25.63.10      x.acme.com         # x client host
18
19 # localhost name resolution is handled within DNS itself.
20 #      127.0.0.1        localhost
21 #      ::1              localhost
22 192.168.1.68           notification.local
23
```

Figure 17: Host File edit

✓ GTA V	Public	Yes	Allow	No
✓ iTunes.MSI	All	Yes	Allow	No
✓ notify	All	Yes	Allow	No
✓ Python	Private...	Yes	Allow	No
✓ Python	Private...	Yes	Allow	No

Figure 18: Firewall Port Open

5.3 Implementation of Core Functionalities

Secure File Sharing

Encryption Process

In order to secure sensitive medical information during transmission, the encryption procedure is a crucial component of the product. The process is described in greater depth below:

When it's come to the User Interaction part The encryption software is used to start the process when a young doctor wants to exchange medical data with a senior doctor.when the file selection screen appears The user chooses which particular medical files they want to share inside the encryption program. These files could include patient information, images of medical treatments, or reports. Using encryption After that, the chosen files are encrypted using a powerful encryption method, such as Advanced Encryption Standard (AES). The files' contents are scrambled into an unreadable format by the encryption process using the selected random key. This guarantees that the original data can only be decrypted and accessed by those who have the proper random key. The program creates an encrypted version of the chosen files after encryption, guaranteeing the confidentiality of the data during transport.

Uploading Files Using WinSCP

WinSCP, which uses SFTP (Secure File transmission Protocol) for secure file transmission, may upload the medical files to the server once they have been safely encrypted:

WinSCP Configuration part for the users set up WinSCP with the required connection information, such as the server's IP address or hostname, port number, and their login credentials, prior to uploading. Users connect to the server after setup by pressing the "Connect" button on the WinSCP interface. Users find the encrypted medical files on their own PC by navigating there. Following that, users may drag and drop the encrypted files from their local workstation into the relevant directory on the server. As an alternative, users may choose and upload the files using the WinSCP "Upload" feature. WinSCP makes use of the SFTP protocol, which encrypts the files while they are in transit to keep them private. When it comes to the storage of files the user's profile on Windows Server 2019 maintains a hierarchical structure for the uploaded files, ensuring organized access.

Downloading Files Using WinSCP

Senior doctors obtain and decrypt medical materials supplied by Junior doctors by downloading them using WinSCP. Senior doctors, like Junior doctors, set up WinSCP with the required connection information, such as the server's IP address or hostname, port number, and login credentials. Senior physicians connect to the server by clicking the "Connect" button in the WinSCP interface after setting WinSCP. They access the directory containing the encrypted medical files after connecting. After that, senior physicians may choose the encrypted data they want to download from the server. The encrypted files are transferred securely to their local device by either dragging and dropping the files to their computer or by utilizing the "Download" feature of WinSCP. Senior physicians then use encryption and decryption software to choose encrypted files that they can then decode to retrieve the original medical records.

Integrity Monitoring

Integrity monitoring is a crucial part of the project since it guarantees the confidentiality and reliability of medical records exchanged among doctors. The production of hash values, ongoing monitoring, and alerting systems are all covered in this section's thorough explanation of the integrity monitoring system's operation.

Hash Value Generation

Every medical file submitted to the Windows Server 2019 platform is given a cryptographic hash value, which is the foundation of the integrity monitoring system. The following actions are performed during the hashing process:

As soon as a junior doctor uploads a document or file, the integrity monitoring system generates a cryptographic hash value for that document or file. A unique digital fingerprint of the contents of the file is represented by this hash value. When creating the hash values, the authors use a trusted and well-known hash method (such SHA-256). As a consequence, trustworthy hash results are generated securely and quickly. However, to increase the security of the hash, the authors have added multiple rounds of SHA-256 hash generator.

On the server, the matching file and the produced hash values are safely kept together. This connection makes it possible to compare and confirm afterwards. The integrity monitoring system constantly runs checks on the hash values that are saved in order to preserve the integrity of the shared medical information. The steps in the continuous monitoring process are as follows:

All files that are saved have their hash values updated by the system. For each file, the newly generated hash values are compared to the hash values that were previously saved. The impacted file is instantly marked as possibly compromised if the system discovers any inconsistencies between the hash values that were recalculated and those that were previously saved. This can be the consequence of data corruption, unauthorized changes, or removals. Figure 18 represent the integrity monitoring window for the admin.

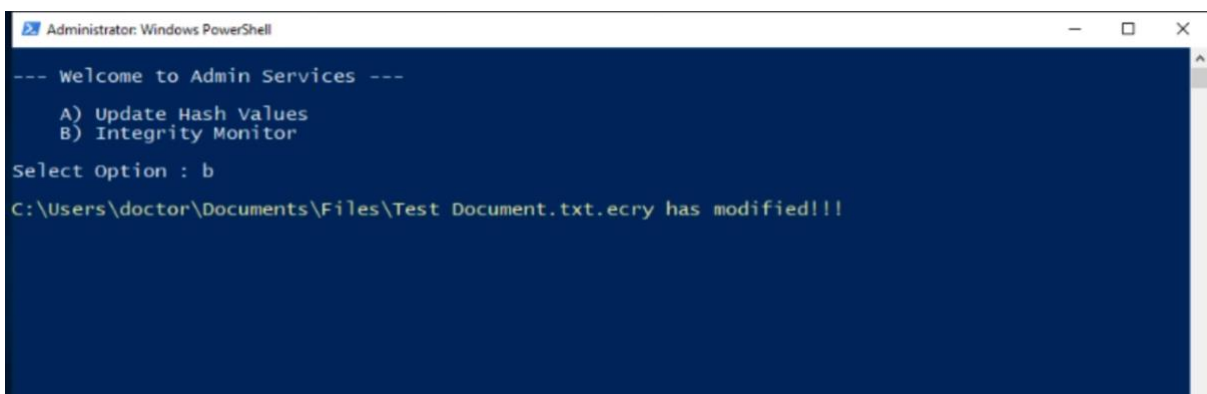


Figure 19: Admin side integrity monitor

Message Communication

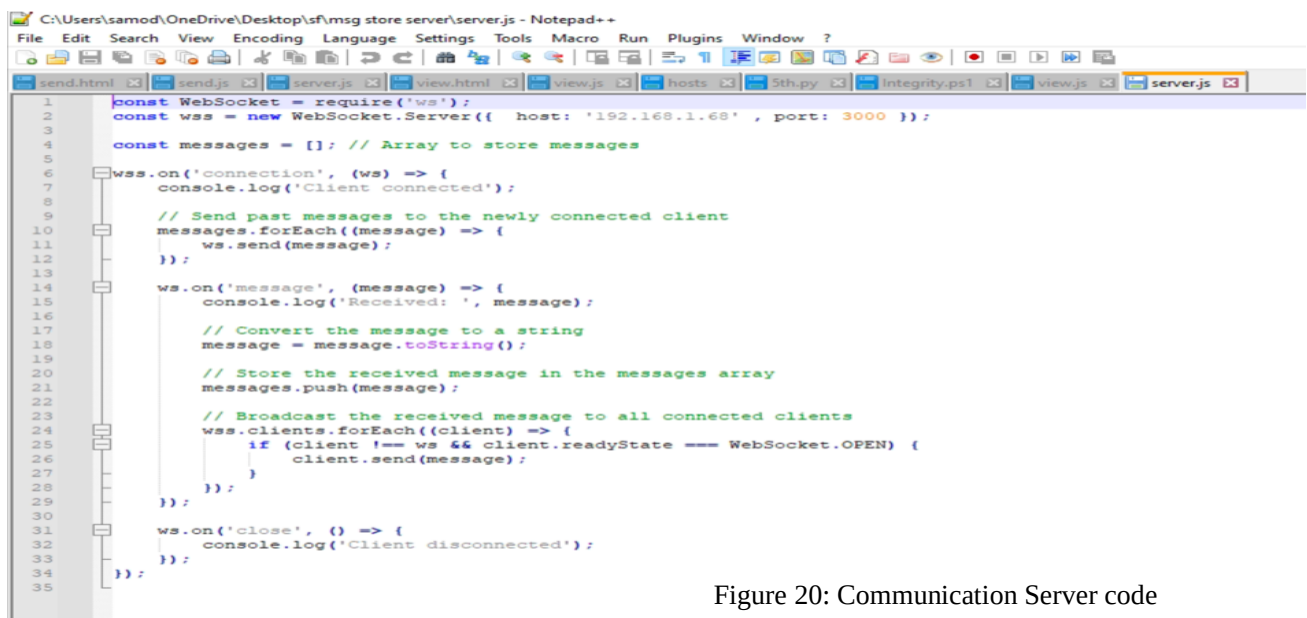
Hosting on IIS

The authors created a messaging system that is hosted on the Internet Information Services (IIS) infrastructure to enable smooth communication between medical professionals. The authors were able to develop a user-friendly web-based message exchange interface thanks to the choice of infrastructure. Here are some of the main parts and features in more detail.

On the IIS server, the authors created and hosted two HTML files. The user interface for the messaging system is made up of these HTML files. One file is used to create and transmit messages, while the other is used to show messages that have been received. Through the online interface, users can quickly create messages. Once written and delivered, a message is then stored on the server. In order to facilitate effective retrieval and presentation, messages are categorized and tagged with information. Each message has a sender, a receiver or recipients, a timestamp, and the message's content.

server.js File

The server.js file is essential for managing the messaging system's backend operations. The Windows Server system runs this JavaScript program which is in charge of numerous crucial features. Receiving Messages is like the server.js module gets the message data from the web server, verifies it, and then processes it for storage once a user submits a message using the HTML interface. Messages Storing in an Array The authors created an array-based message storage system to make sure users can access older messages. Each message is placed into this array as soon as it is received. Regardless of whether the user views the notification tab, this array acts as a repository for all sent and received messages. The array's messages are metadata-tagged, making it simple to retrieve and display them in the user interface. Even if they haven't recently actively visited the notification page, users may always see their message history. Figure 20, shows communication server code created.

The image shows a Notepad++ window with the file path 'C:\Users\samod\OneDrive\Desktop\sf\msg store server\server.js'. The code is as follows:

```
1  const WebSocket = require('ws');
2  const wss = new WebSocket.Server({ host: '192.168.1.68', port: 3000 });
3
4  const messages = []; // Array to store messages
5
6  wss.on('connection', (ws) => {
7    console.log('Client connected');
8
9    // Send past messages to the newly connected client
10   messages.forEach((message) => {
11     ws.send(message);
12   });
13
14   ws.on('message', (message) => {
15     console.log('Received: ', message);
16
17     // Convert the message to a string
18     message = message.toString();
19
20     // Store the received message in the messages array
21     messages.push(message);
22
23     // Broadcast the received message to all connected clients
24     wss.clients.forEach((client) => {
25       if (client !== ws && client.readyState === WebSocket.OPEN) {
26         client.send(message);
27       }
28     });
29   });
30
31   ws.on('close', () => {
32     console.log('Client disconnected');
33   });
34 });
```

Figure 20: Communication Server code

Ensuring Message Availability

The messaging system's capacity to archive communications and make them accessible is one of its primary benefits. This capability is especially useful in the context of healthcare, as previous correspondence often provides crucial patient information. The authors guarantee that users may access and refer to earlier communications at their leisure, boosting communication and cooperation among medical experts, by storing messages in an array and controlling alerts.

In conclusion, the messaging system provides a reliable and accessible platform for medical professionals to safely interact. It is hosted on IIS and driven by the server.js file. Users are informed of new messages even if they do not actively check the notification tab thanks to notifications and the array-based storage of messages that provides a historical record of communication. Communication in the medical setting is now much more reliable and efficient thanks to its application.

5.4 Chapter Summary

In Chapter 5 precisely navigates the technological choices critical to the application. Python is selected for encryption, praised for readability and a rich library ecosystem. Advanced Encryption Standard (AES) is chosen for superior security and efficiency, emphasizing a 256-bit key. Integrity Monitoring via PowerShell ensures data reliability, while SFTP on Windows Server 2019 prioritizes enhanced security and compatibility. Message Communication utilizes JavaScript and Server.js for real-time capabilities. The implementation of core functionalities,

including secure file sharing and message communication, is detailed. Overall, the chapter strategically establishes a secure and robust foundation for the efficient exchange of medical records among healthcare professionals.

Chapter 6: Testing

6.1 Chapter Overview

This chapter is about the testing phase where the developed software is checked its functions and non functions are working as expected. In this chapter the authors compare the outcome with the expect outcomes. If anything functionality lacks they are to be fixed to get the best results. Functional and Non-Functional testing are included with screen shot.

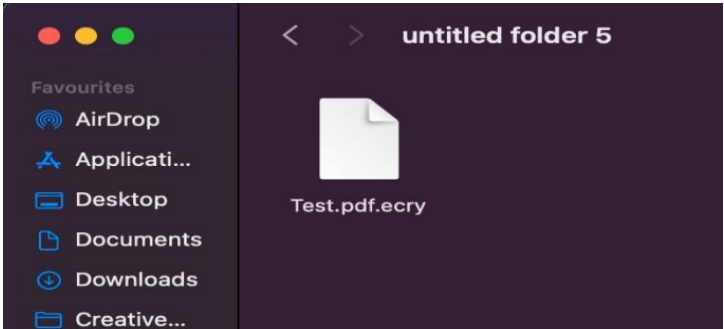
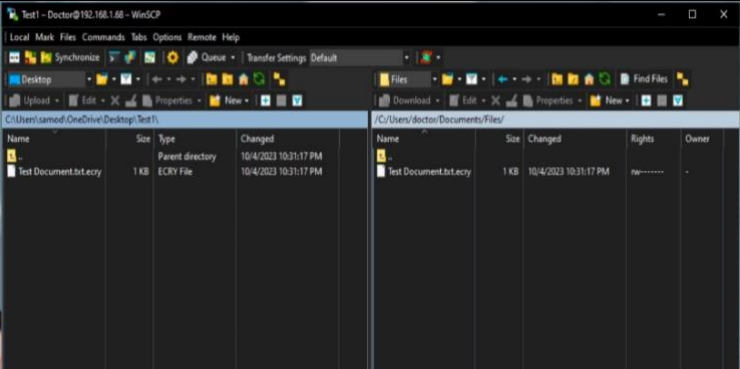
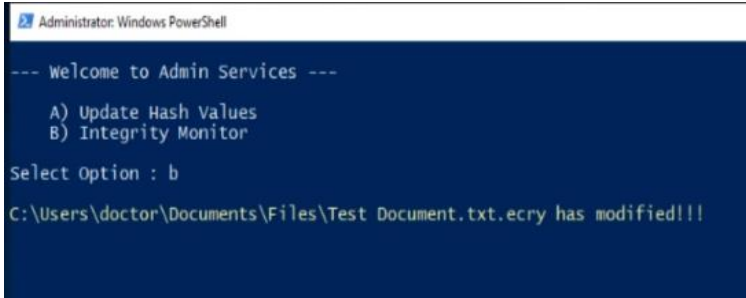
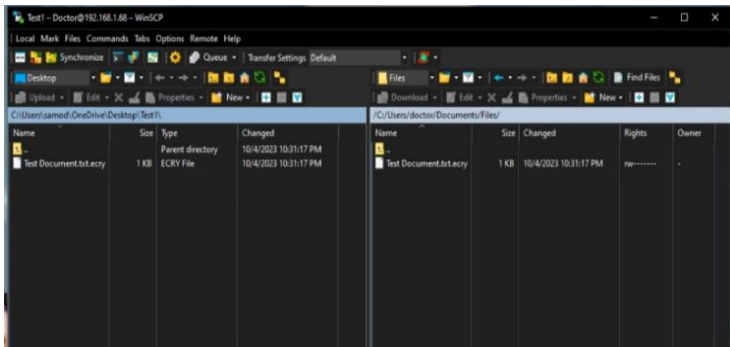
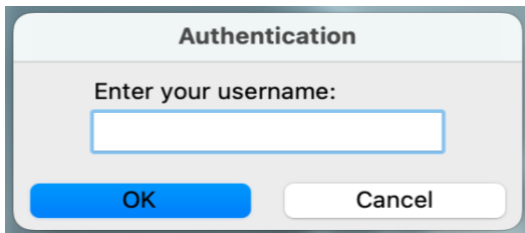
6.2 Objectives and Goals of Testing

Encryption/ Decryption and Integrity Monitoring software is not a freshly introduced concept but there are main objectives that it should full fill. As this is based on medical sector it is more important to achieve the main objectives, so this testing is mainly focused on testing functional requirements.

- Encryption/Decryption and Integrity Monitoring
- Secure File Transfer
- User Access Control
- User Authentication

6.3 Functional Testing

For the software testing purpose, the authors are using black box testing. Black box testing is a special technique for testing software. The software is treated as a black box where the authors are solely focus on the inputs and outputs as well as the functionality of the software.

Function	Expected Outcome	Actual Outcome
Encryption	The files should be encrypted, and their extension should be changed to .encry	
Decryption	The encrypted files should be decrypted, and the extension should be changed back to original extension	
Integrity Monitoring	Whatever happens to the data in the specific folder, the admin should get a notice about that	
User Access Control	The users are not allowed to access other folders except for the specific folder that is separated for this process	
User Authentication	The user must enter valid credentials to login to the software	

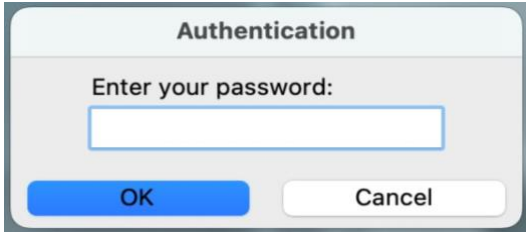
		
--	--	--

Table 8: Functional Testing Results

6.4 Non-Functional Testing

The nonfunctional requirements like performance, reliability and resource constraints are tested and following are the results of them.

Performance – To evaluate the application’s performance, the authors conducted a testing comparison with another encryption algorithm of 3DES by using 10,000 files. The results are presented below

Encryption Algorithm	Encryption (Seconds)	Decryption
AES	0.041024s	0.076999s
3DES	0.225547s	0.448328s

Table 9: Application Performance Testing Results

However, according to the above test results, it is evident that AES, outperforms 3DES in terms of both encryption and decryption efficiency. As a result, this will enable doctors to encrypt and decrypt patients reports swiftly, saving valuable time.

Reliability – The system notification page runs 24 hours a day, if there’s any message from the server admin about the integrity of the files there a notification in The software, it will appear all the time when the user opens the software.

Resources Constraints - The software can even run in window 8 above with a 2GB ram installed.

6.5 Chapter Summary

In this chapter the authors had discussed the expectations and the results by giving you proof of the results. Testing stage helps to prove the software while its in the developing stage. The authors used black box testing method which made the team easy to understand whether the core functions of the software works properly.

Chapter 7: Conclusion

7.1 Chapter Overview

In this concluding chapter, the authors reflect on the journey of The final year project, emphasizing the major accomplishments, lessons learned, and areas of growth. It is the conclusion of the team's work, demonstrating The combined commitment to developing a meaningful and significant project. This overview will cover the achievements of the research goals and objectives, how the authors utilized the course material, the application of unit's learning outcomes, utilization of existing and newly acquired skills as well as the issues encountered, and potential future enhancements.

7.2 Achievements of Research Aims & objectives

The research goals and objectives have been accomplished as a result of The constant dedication to this project. The main aim when starting up was to design, create, and test a hospital-specific Encryption/Decryption Application combined with an Integrity Monitor. By guaranteeing the security and integrity of private medical information as it is being transmitted between experts and doctors, this innovative system has completely changed the way that patient records are protected. The authors built a solid basis for The project through careful stages including problem identification, thorough literature analysis, and complete requirement elicitation. The authors focused on complex encryption and sharing requirements within the medical setting by modeling requirements using advanced analytical design diagrams. The dedication to secure communication in the medical sector has been strengthened by the creation of secure software that makes use of advanced AES and symmetric encryption techniques. The solution's comprehensive strategy has been strengthened by the implementation of an integrity monitoring system, the setting up of a secure SFTP server environment, and the careful

execution of user account management and authentication measures. The thorough testing of encryption methods, and effective integration of The Encryption/Decryption Application with the SFTP server are just a few examples of the meticulous attention to detail that went into this project. The authors are currently at the forefront of offering a ground-breaking solution that tackles the problems encountered by the healthcare industry, putting the well-being and care of patients above all else, thanks to extensive documentation and a smooth deployment procedure. This achievement marks a significant milestone in advancing the confidentiality and integrity of medical data, setting a new standard for secure medical data exchange.

7.3 Utilization of Knowledge from the Course

The knowledge gleaned from The course played an important role in the understanding of The project objectives. From understanding the dynamics of team formation to the careful process of project selection, every step was informed by the insights acquired. The course equipped the team with the essential skills to craft a comprehensive project proposal, ensuring that The goals and methods were well-defined. As the authors explored into the development phase, the principles and techniques communicated by the course were fundamental in guiding The progress. Moreover, the expertise gained in presentation techniques supported The ability to articulate The ideas effectively. This comprehensive utilization of course knowledge empowered the team to successfully design, develop, and evaluate the Encryption/Decryption Application and Integrity Monitor, achieving the primary aim of safeguarding sensitive patient data within the hospital setting.

7.4 Use of Existing Skills

The successful installation of The hospital application was largely due to The prior experience in software development and project management. A strong project plan with clearly defined milestones and dates could be created thanks to the team's experience in project management. While giving the team leader a way to properly track each team member's progress, this structure made it simpler for team members to complete their jobs. The authors made use of a variety of existing abilities and knowledge when creating The hospital application. The authors were able to develop a user-friendly interface that eased the process of encrypting, decrypting, and sharing medical and patient records thanks to the Python programming language expertise

of The team. Additionally, the authors were able to build a safe and reliable architecture for data management within the hospital's network thanks to The existing experience with Windows Server 2019, WinSCP, and SFTP. This expertise with the tools and technology was important to the execution of The project successfully.

7.5 Use of New Skills

In the process of creating The application, the authors set out on a quest to learn and put several fresh and sophisticated abilities to use. These newly acquired skills were crucial in raising the project's functionality, security, and general level of quality.

1. GUI Design with Python:

The authors used Python to create a graphical user interface (GUI), which is one of the project's most notable accomplishments. The transition to front-end design was a substantial one for The team, even though the authors had solid credentials in back-end development and data administration. The authors heavily rely on Python libraries like Tkinter to achieve this. The user experience with the application was revolutionized by the intuitive and aesthetically pleasing interface the authors were able to create thanks to these libraries. Doctors and authorized workers could now easily traverse the system, choose which files to encrypt, decrypt them as necessary, and securely communicate medical records—all within a fluid and enjoyable environment. The capabilities have expanded because of the GUI design skills the authors've acquired, and opportunities to develop user-centric user interfaces for future projects have now become possible.

2. Advanced Encryption Techniques with AES:

The hospital application project gave the team a priceless chance to learn more about sophisticated encryption methods, particularly those that use the Advanced Encryption Standard (AES). It was crucial to provide the highest level of security and privacy for patients and medical records. To do this, the authors looked into Python libraries like those for cryptography, which not only made it easier to implement AES but also made it possible to maintain encryption keys in a secure manner. AES is well known for its power and dependability in safeguarding sensitive data, and The use of it greatly increased the application's security. As a result of The team's newly acquired AES

encryption knowledge, the authors are now better prepared to handle data security issues in a variety of fields. The authors now have a strong cybersecurity tool thanks to this experience, which will help the team protect sensitive data in the future and guarantee its confidentiality and integrity.

3. Python Libraries for Enhanced Functionality:

The creative placement of Python modules to improve several functionalities was a crucial component of The project's success. These libraries served as multipliers, greatly enhancing the application's functionality and performance. To help with secure file transfers using the Secure File Transfer Protocol (SFTP), for instance, the authors simply integrated WinSCP. This integration improved the security of data transmission while also streamlining the procedure and making it easier for staff and medical professionals to use. The libraries the authors utilized were also used to interface successfully with the Windows Server 2019 operating system, ensuring compatibility and a seamless user experience within the Windows environment.

7.6 Achievements of Learning Outcomes

The accomplishment of the learning objectives for The unit is evidence of The teamwork and creative use of The knowledge. Through this project, the authors showed that the authors can carefully prepare and precisely define the scope. A key component of The strategy was the concise explanation of the project's goals, risks, and benefits. The authors demonstrated The competence in carrying out activities of industrial importance by effectively performing a project that adheres to industry standards. Furthermore, The dedication to responsibility and transparency has been strengthened by a thorough review and open sharing of project findings. These accomplishments confirm that the authors not only fulfilled but also beyond the learning goals for The course, displaying a high degree of project planning, execution, and communication.

7.7 Problem and Challenges Faced

As the project progressed, the authors encountered several issues that required creative solutions, especially because every member of the team was a cybersecurity student. This distinct backdrop gave The project's development depth and intricacy.

1. Integration of User Interface (GUI):

One of the primary issues the authors ran into was integrating the Python-based user interface with the pre-existing application pages. The cumulative experience in cybersecurity made it challenging for the team to successfully develop applications and create user interfaces (GUIs), two fields that were relatively new to the team. To provide a smooth transition between these components, extensive coding and debugging efforts were required in this case. The learning curve was steep, but the authors were able to balance The understanding of cybersecurity with the requirements of GUI design.

2. Encryption Key Management and Access Permissions:

Because of The expertise in cybersecurity, the authors are especially aware of the value of effective data protection. The issue was in translating this awareness into actual practice. It was a complex process to manage encryption keys and access rights for different users within the application. The cybersecurity knowledge gave the team a strong basis to undertake this assignment with a greater awareness of security issues. Nevertheless, in order to handle the complexity of user access control, the authors had to adapt and broaden The skill set. This technique allowed the team to find a compromise between strict security precautions and user-friendly access, all while maintaining the system's integrity. It was a difficult but ultimately gratifying task.

7.8 Deviations

During the Developing Process there were 3 main deviations in the Application. There are,

1. Integrity Check Mechanism
2. GUI Architecture
3. Additional use of Programming languages

Integrity Check Mechanism

Initially, the authors intention was to assess integrity within the application. The initial Planed GUI featured an “Integrity” Option, allowing doctors to select a preferred file and click the “check integrity” button. Upon execution, the application would verify the integrity and a notification would be displayed within the application stating the status of the integrity of the file whether Integrity pass or fail. However, this process has serval drawbacks. It’s a highly manual and demands significant effort, especially for a non-IT professional like doctors. This manual nature is very time consuming and given the busy work Schedules of doctors, there is a high-risk doctor may forget to check the files integrity at certain times.

To avoid this situation, the authors proposed shifting the responsibility from doctors to an assigned admin role with authorization to verify the integrity of transmitted files. The admin would consistently monitor file integrity, and in the event of any integrity misconduct, a detailed alert would notify the admin, providing information such as the name of the file and the cause. Finally, the admin could send a notification for the respective doctor regarding the files’ integrity status. However, the, technical aspect of the Integrity monitor remained same as discussed in the [5.2.3 Integrity monitor.](#)

GUI Architecture

The initial application GUI planned was a very simple and straight forward. It had on two 2 windows such as the login and the options. Figure 12, Figure 13 are the Initial developed GUI of the application.

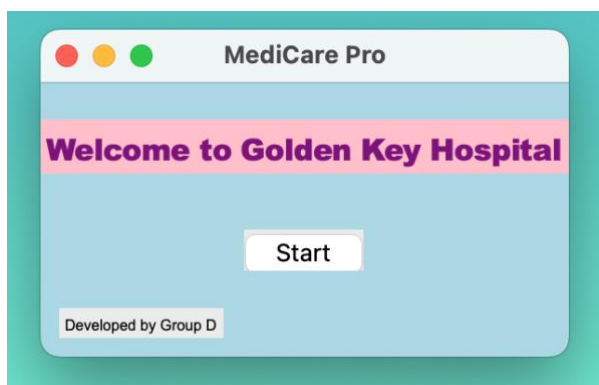


Figure 21: Initial Welcome Window

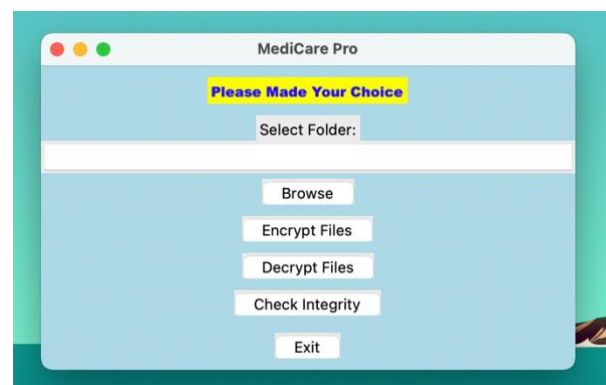


Figure 22: Initial Option Window

However, while in the developing phase the authors identified an opportunity to enhance the user friendliness and elevate it to an organization level by introducing more windows or interfaces to the application. The challenging part was to maintain the user-friendliness and the simplicity as the application is used by Doctors who are Non-IT professionals. [4.4.3 UI Design](#), showcases the Final Developed GUI of the application.

Additional Use of Programming languages

The authors choose Python as the exclusive programming language to complete the entire application process. While the Full GUI was built using Python, HTML was employed for hosting user files, JavaScript facilitated the transmission of the notifications to doctor's side, and PowerShell as utilized for the integrity Monitor.

7.9 Future Enhancements

There are several Future enhancements the team plan to the MedicaCare Pro existing application. There are,

1. **User personalization:** User personalization is another plan the team wish to add in the future, as it allows doctors to personalize their experience such as changing themes or rearrange elements.
2. **Admin Notification:** Currently even though there is an option as "Notifications from admin" in the application, no notification is received. Doctors have to manually check whether they have received any messages from the admin. The plan is to develop a notification system where, upon receiving a message from the admin, a windows notification will be appeared as well as a new message will be indicated in the Notification from admin button.

However, apart from these two main future enhancements, there are numerous other opportunities to improve the hospital application in the future. Priority one is to strengthen security measures by investigating cutting-edge encryption methods and authentication procedures. By adding features like real-time collaboration, automated reminders for appointments and updates, and sophisticated reporting tools, increase the application's functionality. The team also hope to make the process much smoother and expand the GUI for all the options in the application. The application will continue to be at the forefront of patient

record protection and effective healthcare administration by keeping up with developing technology and best practices in data management for the healthcare industry as the authors work toward achieving these objectives.

7.10 Chapter Summary

The chapter 7 showcase the successful development of the application. The chapter delves into the team's dept use of existing skills, used of new skills such as GUI creation with Python and challenges faced during the development. Three main deviations from the initial plan supported by evidence snapshots were presented. Finally, the chapter concluded with insightful reflection on future enchantments to increase the security and the user friendliness, the proposed improvements include the addition of a new feature of user personalization, admin notification and increased security measured.

8. Teamwork

Throughout the project development and execution process, the team showcased a truly collaborative spirit that drives to its success. Each individual worked seamlessly, by actively participating and bringing their own unique skills to the table. The approach was to not only to work together but leveraging the diverse of skills within the team to create a robust and effective encryption / decryption application tailored for the healthcare.

Not only Collaborative work, but Communication also played a vital role in steering the project towards its completion. Frequent and clear communication were established paying the wave for an environment where team members seamlessly shared their ideas and knowledge. Regular meetings with both team members and supervisor ensured that everyone was on the same page, minimizing misunderstandings, correct errors and doubts, and enhancing the overall project efficiency of the development process. Table 10, Table 11 represent the evidence of the communication in the form of meetings with supervisor and team members. The details of the meetings including the date, duration and topics discussed are mentioned.

Meetings with Supervisor

Date (MM/DD/YYYY)	Duration	Description
08/06/2023	30 Min	First Meeting with the Supervisor. Introduced the Team members and presented the Project proposal
08/15/2023	1 hour	Modified the changes from the first meeting. Got the Feedback for The Assignment 01 Project Proposal Report.
09/20/2023	1 hour 3 Min	Presented the 1 Prototype of the application.
10/02/2023	53 Min	Presented the Second Prototype of the application and Discussed about the Presentation
10/07/2023	1 hour 15 Min	Mock Presentation

Table 10: Meetings evidence with Supervisor

Meetings with Team Members

Date (MM/DD/YYYY)	Duration	Description
07/28/2023	40 Min	First Meeting with the team members. Introduced The self and Planned a Project.
08/06/2023	1 hour 35 Min	Made changes to the proposed project after the meeting with supervisor
08/08/2023	40 Min	Divided the work for the Assignment 01 (Project Proposal report)
08/12/2023	1 hour 30 min	Discussed about the Issues and Problems about the proposal.
08/16/2023	30 min	Finalized the Project Proposal and divided the work to build the application
08/27/2023	40 min	Progress meeting. Discussed the issues of having while developing the application.
09/10/2023	2 hour 20 min	Combined each work and build the first prototype
09/13/2023	50 min	Trouble Shoot
09/17/2023	1 hour 40 min	Tested the first prototype
09/20/2023	30 min	Divided work to modify the application
09/30/2023	1 hour	Combined the Prototype 2 and tested. Divided work for the Final Report
10/02/2023	30 min	Discussed about the presentation and divided work
10/04/2023	40 min	Combined all the presentation slides and make few changes
10/09/2023	1 hours 30 min	Conducted a Practice Presentation
10/11/2023	30 min	Quick meeting to discuss Progress On the report

13/11/2023	45 min	Discussed issues of Diagrams
17/11/2023	1 hour	Finalized the final report

Table 11: Meeting evidence with Team members

Professionalism is another aspect of teamwork, and it is the foundation of the success. From the start to finish of the project, every team member was committed, well prepared and punctual. As per the above-mentioned tables it is evident that meetings were conducted with a purpose, and especially the deliverables were made on time. However, there were sometimes and situations when things didn't go as planned, the team's professionalism shone through. Several team meetings were canceled due to the busy work schedules, and some individual work submit deadlines was missed. However, the respective team member always informed the team of the situation in advance and provided explanations. Additionally, alteration solutions were proposed to reschedule meetings and complete work on time. None of team members pointed finger, instead everyone worked together as a team to find solutions and learn from the experiences. This demonstrates the high level of professionalism and commitment of each members to the team and the project. As a result, this enabled everyone to gain trust and confident among team members ultimately led to the project success.

8. References

- Gautam, P., Ansari, M. D., & Sharma, S. (2019). Enhanced security for electronic health care information using obfuscation and RSA algorithm in cloud computing. *International Journal of Information Security and Privacy*, 13(1), 59–69.
<https://doi.org/10.4018/ijisp.2019010105>
- GeeksforGeeks. (2023). Difference between MD5 and SHA1. *GeeksforGeeks*.
<https://www.geeksforgeeks.org/difference-between-md5-and-sha1/>
- HENDI, A. Y., Dwairi, M., Al-Qadi, Z. A., & Soliman, M. S. (2022). A novel simple and highly secure method for data encryption-decryption. *International Journal of Communication Networks and Information Security (IJCNIS)*, 11(1).
<https://doi.org/10.17762/ijcnis.v11i1.3999>
- Iqbal, S., Kiah, L. M., Zaidan, A. A., Zaidan, B. B., Albahri, O. S., Albahri, A. S., & Alsalem, M. A. (2018). Real-time-based E-health systems: design and implementation of a lightweight key management protocol for securing sensitive information of patients. *Health and Technology*, 9(2), 93–111. <https://doi.org/10.1007/s12553-018-0252-4>
- Latinov, L. (n.d.). MD5, SHA-1, SHA-256 and SHA-512 speed performance – Automation Rhapsody. https://automationrhapsody.com/md5-sha-1-sha-256-sha-512-speed-performance/#google_vignette
- Nakamura, R., & Kuga, Y. (2023). Multi-threaded scp: Easy and Fast File Transfer over SSH. *Practice and Experience in Advanced Research Computing*.
<https://doi.org/10.1145/3569951.3597582>
- Orobosade, A., Aderonke, T., Boniface, A., & J., A. (2020). Cloud Application Security using Hybrid Encryption. *Communications on Applied Electronics*, 7(33), 25–31.
<https://doi.org/10.5120/cae2020652866>

- Sani. (2022). An Improved Cryptographic Scheme Using AES & RSA Algorithms for Maximum Security in File Encryption and Decryption. *International Journal of Scientific Development and Research*, 7. www.ijedr.org
- Satar, S. D. M., Mohamed, M. A., Hussin, M., Hanapi, Z. M., & Satar, S. D. M. (2021). Cloud-based Secure Healthcare Framework by using Enhanced Ciphertext Policy Attribute-Based Encryption Scheme. *International Journal of Advanced Computer Science and Applications*, 12(6). <https://doi.org/10.14569/ijacsa.2021.0120643>
- Security, T. (2022). AES vs DES Encryption: Why Advanced Encryption Standard (AES) has replaced DES, 3DES and TDEA. *Precisely*. <https://www.precisely.com/blog/data-security/aes-vs-des-encryption-standard-3des-tdea>
- Shahzadi, R., Anwar, S. M., Qamar, F., Ali, M., & Rodrigues, J. J. P. C. (2019). Chaos based enhanced RC5 algorithm for security and integrity of clinical images in remote health monitoring. *IEEE Access*, 7, 52858–52870. <https://doi.org/10.1109/access.2019.2909554>
- Sumagita, M., & Riadi, I. (2018). Analysis of Secure Hash Algorithm (SHA) 512 for encryption process on web based application. *International Journal of Cyber-Security and Digital Forensics*, 7(4), 373–381. <http://sdiwc.net/digital-library/analysis-of-secure-hash-algorithm-sha-512-for-encryption-process-on-web-based-application>